Exloris Technical Documentation

Project architecture, scripts, gameplay, dependencies, and diagrams

Unity 6000.6.0f1 • Static project analysis

This document combines the project documentation and includes clickable navigation.

Contents

- Exloris — project overview
  - Purpose and gameplay
  - Verified project facts
  - Main route
  - High-level architecture
  - Major systems
  - Limitations and open issues
- Project structure
  - Root layout
  - Project-owned code organization
  - Scene files
  - Generated, imported, and authored content
  - Inspection limits
- System architecture and connections
  - Architecture at a glance
  - Ownership and communication
  - Direct and indirect dependencies
  - EventBus behavior
  - Architecture boundaries and coupling
  - Initialization and lifecycle considerations
- Script reference
  - Core data and messaging
  - Boat level gameplay
  - Boat presentation and shared presentation
  - Player, persistence, and scene management
  - Nilometer, audio, localization, and legacy scripts
  - Editor script
  - Script usage caveats
- Gameplay lifecycle
  - Evidence labels
  - 1. Application startup and entry scene
  - 2. Menu, player selection, and map route
  - 3. Boat level initialization
  - 4. Starting and steering
  - 5. Main boat loop and collisions
  - 6. Pause, help, and settings
  - 7. Arrival and boat completion
  - 8. Nilometer learning activity
  - 9. Audio, UI, and visual feedback
  - 10. Persistence, restart, and exit
- Scenes and transitions
  - Build scene list
  - All checked-in scene files
  - Navigation mechanisms
  - Scene-local setup references
  - Profile and runtime limitations
- Data and asset dependencies
  - Data contracts and persistence
  - Localization and audio content
  - Rendering, animation, and UI assets
  - Prefabs
  - Runtime asset loading findings

- Input and physics

External packages and integrations
- Relevant Unity packages
- Imported plugins and source content
- Addressables and localization integration
- Platforms and build integration
- No evidence found

Setup, build, and configuration
- Open the project
- Build scene configuration
- Input configuration
- Rendering and platform settings
- Required scene wiring checks
- Build and verification checklist

Editable flowcharts
- A. Project overview
- B. Startup and initialization
- C. Main gameplay loop — boat level
- D. Scene transitions
- E. Typed gameplay event flow
- F. Victory, failure, and restart
- Additional learning and persistence flows

UML-style diagrams
- 1. Boat system class diagram
- 2. Nilometer class diagram
- 3. Persistence class diagram
- 4. Boat state diagram
- 5. Sequence diagram — boat obstacle hit
- 6. Sequence diagram — Nilometer answer
- 7. Sequence diagram — save/load
- Diagram limits

Glossary and technical notes
- Glossary
- Important technical notes
- Scope and limitations
- Suggested follow-up verification in Unity
- Class Diagram
- Gameplay Flow
- Project Overview
- Scene Transitions
- Script Dependencies
- Sequence Diagrams
- Startup Flow
- System Architecture

Exloris — project overview

Purpose and gameplay

Exloris is a Unity 2D educational game set around travel through ancient Egypt and a learning activity built around the Nilometer. The serialized build list contains a main menu, player selection, map, boat level, and Nilometer scene. The boat level is a three-lane forward journey with obstacles, coins, stars, an arrival sequence, and a completion panel. The Nilometer experience presents staged water levels, spoken dialogue, multiple-choice questions, hints, lesson content, and a completion display.

This description is based on the current scene names, project code, and serialized configuration. It is not a claim that every route has been play-tested.

Verified project facts

Item	Evidence / value
Unity editor version	6000.6.0f1 from ProjectSettings/ProjectVersion.txt.
Render pipeline	Universal Render Pipeline package 17.6.0; project includes a URP asset and 2D renderer assets.
Language	C# scripts under Assets/scripts and Assets/Editor.
Input	Input System package 1.20.0; boat input directly uses UnityEngine.InputSystem. The project also contains legacy Input Manager axes.
Localization	Unity Localization 1.5.12; English and Arabic locale/table assets are present. RTL TextMeshPro support is imported.
UI	UGUI, TextMeshPro, UnityEvents, and custom presentation scripts.
Build scene entries	7 configured entries: 5 enabled and 2 disabled. The two disabled Exloris scene paths are missing in the current checkout.
Player save	Application.persistentDataPath/player.save, written by Exloris.Saving.SaveSystem.
Addressables	Addressable settings and localization groups exist. No Addressables runtime load call was found in the inspected project-owned C# scripts.
Target-related configuration	Android, iPhone, Standalone, and WebGL settings/defines are present. This does not establish a supported release matrix or successful builds.

Main route

The enabled scene order in ProjectSettings/EditorBuildSettings.asset is:

main menu → player → Map → boat/BoatLevel → NiloMeter/NiloMeter

The scene order is confirmed. Actual button navigation is serialized in scenes and exposed through menu/scene-manager components; not every edge was verified by running the game. See scene and transition reference.

High-level architecture

The project currently combines two styles:

- The boat level has a newer namespaced feature structure (`BoatGame.GameplaySystems`, `BoatGame.Presentation`, `BoatGame.CoreData`). `BoatLevelFlow` owns level state and score; input and boat actors report interaction; a typed static `EventBus` informs UI, audio, and VFX.
- Menu, player selection, Nilometer, and older boat scenes use mostly global-namespace, scene-bound `MonoBehaviour`s and UnityEvents. Several old and new classes share names, so scene attachment and namespaces matter.

There is no single application bootstrap coordinator in the active build list. `Assets/Exloris/Scenes/Bootstrap.unity` and `FoundationHarness.unity` are disabled entries and do not exist at their configured paths. Unity starts from the first enabled build scene under the normal build convention, but actual launch behavior was not runtime-tested.

Major systems

System	Primary code
Scene navigation	Assets/scripts/SceneManager.cs, ManagersDetector.cs, scene UnityEvents.
Avatar/player	Player/player.cs, PlayerAvatarSelector.cs, SaveManager.cs, selection UI.
Boat gameplay	GameplaySystems/BoatLevel/BoatLevelFlow.cs, BoatController.cs, BoatInput.cs.
Boat presentation	Presentation/BoatLevelUI/BoatLevelUI.cs, dialogue, stars, VFX.
Nilometer	Presentation/SharedUI/NilometerUI.cs, WaterOperator.cs, WaterAnimator.cs, Tail.cs.
Audio	Event-driven AudioService; older key-based SoundManager also remains.
Persistence	Exloris.Saving.SaveManager and SaveSystem.
Localization	Unity Localization tables plus LanguageSwitcher, LocalizationID, and localized audio helpers.

Limitations and open issues

- The configured continuation name in `BoatLevelFlow` defaults to `BoatHistory`, which is not enabled in the build list. `BoatLevelUI.OnButtonClick_Next` currently does not call `LoadNextSection`.
- No game-over or explicit failure state was found in the boat state machine. Stars can reach zero, but no separate fail transition is implemented there.
- The repo has both a global `BoatController` for `Assets/Scenes/boat/Boat.unity` and a namespaced `BoatController` for `BoatLevel.unity`.
- Addressables are configured for localization content, but project-owned scripts inspected here do not call the Addressables API directly.
- A local save key is kept in ignored `SaveSecrets.cs`. Its value is deliberately excluded from these documents. Client-side key storage is not strong anti-tamper protection.
- This analysis is static. Scene runtime order, visual behavior, platform builds, and all serialized UnityEvent effects remain unverified unless stated otherwise.

Project structure

This page describes the checked-out repository layout at a useful system level. It does not inventory every imported art file or third-party example asset.

Root layout

Path	Purpose / observed contents
Assets/Scenes/	9 Unity scenes: 5 enabled build scenes and 4 other scenes/prototypes.
Assets/scripts/	55 project-owned runtime C# files, including gameplay, UI, persistence, localization, and legacy helpers.
Assets/Editor/	Project-owned editor/build helper WebGLBuildCli.cs.
Assets/Exloris/	Newer target-oriented content/runtime/editor folder layout. It contains localization assets and many runtime/core folder placeholders; the active gameplay C# inventory remains primarily under Assets/scripts.
Assets/prefabs/	26 project prefabs found at the time of inventory, including audio, UI, coin, obstacle, island, avatar, and manager-related names. Actual use varies by scene.
Assets/AddressableAssetsData/	Addressable settings, groups, schemas, and builders; localization-related groups are present.
Assets/Exloris/Content/Localization/	English/Arabic locale, string table, audio table, and localization settings assets.
Assets/Animation/	Avatar sprite animation content.
Assets/Audio/	Ambience, music, SFX, and voice content.
Assets/Resources/	DOTweenSettings.asset; this is configuration, not evidence of a project runtime Resources.Load call.
Assets/StreamingAssets/Exloris/Intro/	intro-16x9.mp4 is present. Runtime use was not proven from inspected project-owned scripts.
Assets/Settings/	URP, renderer, and post-processing settings assets.
Assets/Plugins/, Assets/RTLTMPro/, Assets/AnimationSequencer/, Assets/Events 2.0 for Unity/, Assets/Cainos/	Third-party plugins, samples, and imported art/tool content.
Packages/	manifest.json and lock file declare Unity packages and Git package dependencies.
ProjectSettings/	Unity editor, player, graphics, quality, input, and build scene configuration.
Builds/, Logs/, Recordings/, UserSettings/	Local build/log/recording/editor data; not treated as source architecture.

Project-owned code organization

```
Assets/scripts/
  Boat/                    Legacy boat scripts
  CoreData/                Boat events, settings, dialogue, service/event bus
  CoreGameSystems/Audio/   Event-driven audio service
  GameplaySystems/BoatLevel/  Namespaced boat gameplay
```

```
Hover/                    UI/world pointer feedback
Localization/             Locale and instruction helpers
Player/                   Player data and avatar components
Presentation/BoatLevelUI/ Boat UI and feedback
Presentation/SharedUI/    Shared dialogue, options, Nilometer and expressions
Save system/              Save coordinator, serializer, local key provider
audio/                    Legacy SoundManager
```

Scene files

Scene	Build state	Observed role / attached project components
Assets/Scenes/main menu.unity	Enabled	Menu/entry scene. Serialized components include scene organization, scene manager, save manager, legacy sound manager, manager façade, and event-driven audio service.
Assets/Scenes/player.unity	Enabled	Player selection and locale/instruction UI. Includes player/avatar selection, save, localization, and presentation components.
Assets/Scenes/Map.unity	Enabled	Map UI with scene/manager and localization/hover helpers.
Assets/Scenes/boat/BoatLevel.unity	Enabled	Namespaced boat gameplay flow, controller, input, camera, VFX, and boat presentation.
Assets/Scenes/NiloMeter/NiloMeter.unity	Enabled	Water quiz/lesson, dialogue, avatar expressions, stars, save/avatar components, and legacy audio.
Assets/Scenes/boat/Boat.unity	Not in current build list	Older boat implementation with global BoatController, BoatBackground, spawner, score, and hit detector scripts.
Assets/Scenes/boat/BoatHistory.unity	Not in current build list	Title/canvas/camera content. Its name is referenced as the boat flow's default next scene, but it is not an enabled build scene.
Assets/Scenes/SampleScene.unity	Not in current build list	Minimal camera and 2D light scene.
Assets/Scenes/mouseHover.unity	Not in current build list	Hover interaction sample/prototype.

Scene attachment claims above come from matching serialized `m_Script` GUIDs to project script `.meta` files. They do not prove a component is enabled, its references are valid, or its callbacks execute in a built game.

Generated, imported, and authored content

Unity folders such as `Library`, `Temp`, and cache outputs are generated and are not used to infer game architecture. Imported plugin code and samples are distinguished from project-owned scripts. The authored asset catalog and imported content under `Assets/Exloris/Content` indicate an asset organization effort; a directory name alone does not prove runtime use.

Inspection limits

Prefab inventory was performed by file path and serialized script-reference search, not by opening the Unity Inspector. Nested prefab behavior and overrides were not executed. Addressables groups were inventoried, but no Addressables runtime call was found in inspected project scripts.

System architecture and connections

Architecture at a glance

The game is a scene-composed Unity application. Scene YAML and prefabs provide GameObjects, component references, UnityEvent bindings, art, and UI. Runtime code has a newer namespaced boat module alongside older global-namespace scene systems.

```
flowchart TB
  Scenes[Scenes and prefabs] --> Menu[Menu / player / map]
  Scenes --> BoatScene[BoatLevel scene]
  Scenes --> NileScene[Nilometer scene]
  Menu --> Nav[SceneManager and serialized UI callbacks]
  Menu --> Player[Player selection and SaveManager]
  BoatScene --> BoatFlow[BoatLevelFlow]
  BoatFlow --> BoatActors[BoatInput / BoatController / Coin / Obstacle]
  BoatFlow --> Bus[Typed EventBus]
  BoatUI[BoatLevelUI] <--> BoatFlow
  BoatUI --> Bus
  Bus --> Audio[AudioService]
  Bus --> VFX[BoatLevelVfx]
  NileScene --> NileUI[NilometerUI]
  NileUI --> Water[WaterOperator / WaterAnimator]
  NileUI --> Dialogue[Shared DialoguePanel]
  Dialogue --> Bus
  Player --> Save[SaveSystem]
  Content[Localization / audio / sprites / prefabs] --> Scenes
```

This diagram combines code references and serialized scene composition. It does not claim every scene shares one persistent service instance.

Ownership and communication

BoatLevelFlow

Owns / changes: `LevelState`, stars, coins, distance progress, docking.

Trigger and inputs: UI calls (`CompleteIntro`, `StartPlaying`, help/settings/arrival); boat callbacks; frame updates.

Outputs / communication: `LevelStateChangedEvent`, `LevelProgressEvent`, `CoinCollectedEvent`, `BoatHitObstacleEvent`, `PlaySfxEvent`; direct calls into boat/input.

Completion / cleanup: Sets Complete after arrival dialogue; replay reloads active build index; next loads serialized name. `OnDisable` unhooks callbacks and restores time scale.

BoatInput

Owns / changes: Pointer tracking and enabled/disabled input state.

Trigger and inputs: Keyboard and pointer devices via Input System; checks EventSystem raycasts.

Outputs / communication: `OnLaneStep(−1/+1)` C# event.

Completion / cleanup: Clears pointer tracking when disabled.

Namespaced BoatController

Owns / changes: Lane, movement, speed, blocked state, collision feedback, autopilot.

Trigger and inputs: Lane steps and physics trigger contacts; progress from flow.

Outputs / communication: C# obstacle/coin callbacks to flow; `PlaySfxEvent`; `BoatBlockedEvent`.

Completion / cleanup: Stops/autopilots at dock; uses Rigidbody2D movement.

BoatLevelUI

Owns / changes: Screen visibility and HUD rendering.

Trigger and inputs: Button handlers and typed events.

Outputs / communication: Calls level-flow methods directly; publishes tap SFX event.

Completion / cleanup: Removes listeners/subscriptions in lifecycle cleanup.

AudioService

Owns / changes: Audio source playback, fades, volume, snapshots, voice ducking.

Trigger and inputs: Typed SFX/voice/state/settings/pause events.

Outputs / communication: VoiceActivityEvent to expressions.

Completion / cleanup: Unsubscribes in OnDisable; fades use unscaled delta time.

BoatLevelVfx

Owns / changes: Visual particle/camera feedback.

Trigger and inputs: Coin, impact, and level-state events.

Outputs / communication: Particle, camera shake, celebration animation.

Completion / cleanup: Removes event subscriptions in OnDisable.

NilometerUI

Owns / changes: Quiz acceptance, active level/attempts, stars, pending dialogue, lesson/completion visibility.

Trigger and inputs: Dialogue callbacks, water-level event, answer/hint button events.

Outputs / communication: Calls WaterOperator; publishes SFX; invokes serialized UnityEvents.

Completion / cleanup: Removes listeners and callbacks on destroy/disable.

WaterOperator

Owns / changes: Ordered serialized level definitions and current level index.

Trigger and inputs: MoveToLevel, MoveToNextLevel; animation completion.

Outputs / communication: Raises WaterLevelReached UnityEvent-like C# event with index and target.

Completion / cleanup: Stops/guards overlapping level animations.

SaveManager

Owns / changes: Active player and loaded player data.

Trigger and inputs: Player registration, scene/player selection UI, explicit save/load calls.

Outputs / communication: Calls static SaveSystem; invokes serialized OnLoadedData.

Completion / cleanup: Persistent singleton behavior is implemented with DontDestroyOnLoad.

Direct and indirect dependencies

- **Direct compile-time calls:** BoatLevelFlow calls methods on its serialized boat, input, and camera references; NilometerUI calls WaterOperator and dialogue methods; SaveManager calls SaveSystem.
- **Serialized runtime references:** [SerializeField] fields in scene components connect scene objects, buttons, dialogue panels, audio sources, mixer snapshots, water levels, and visual assets. These are not visible from C# alone; scene YAML supplies some evidence.
- **Event-driven dependencies:** EventBus statically routes value-type event structs. It is synchronous and transient; late subscribers do not receive old events.
- **UnityEvent dependencies:** WaterOperator.WaterLevelReachedEvent, answer/completion callbacks, hover callbacks, scene organizer callbacks, and UI button events can invoke targets serialized in scenes. Code inspection alone cannot establish all target bindings.
- **Singleton/global access:** SaveManager, legacy SoundManager, SceneManager, and PlayerManager expose singleton patterns. These introduce global runtime dependencies and duplicate-instance considerations.
- **Runtime loading:** SceneManager calls Unity SceneManager load APIs; spawning instantiates a selected prefab. No project-owned Addressables or Resources load call was found.

EventBus behavior

BoatGame.CoreData.Messaging.EventBus provides a generic static channel per event struct. It avoids a central event dictionary and supports removing a listener during a publish by deferring compaction. Listeners added during a publish run on the next publish. Exceptions from callbacks are not isolated by the bus; an exception can propagate through the publisher. Subscribers should pair Subscribe/Unsubscribe in OnEnable/OnDisable.

Architecture boundaries and coupling

- The namespaced boat system is event-oriented for UI/audio/VFX but retains direct C# callbacks from boat/input actors to BoatLevelFlow.
- BoatLevelUI reaches flow through Service<BoatLevelFlow>, a scene component resolver/cached reference, and also uses typed events for presentation updates.
- Nilometer is more directly coordinated: NilometerUI owns the quiz/dialogue flow and calls water/dialogue/character components directly.
- Audio has two parallel systems. AudioService consumes the typed event bus; SoundManager is a legacy key-based singleton. Their coexistence can create scene-specific differences.
- Global class names collide: two BoatController classes and two DialoguePanel classes exist. PlayerSelector.cs and PlayerUISelector.cs also contain class names opposite their filenames. Preserve exact type and scene binding during maintenance.
- Static references and scene-level singletons are not a demonstrated circular dependency. A cycle was not established from inspected code.

Initialization and lifecycle considerations

Unity does not guarantee ordering among unrelated objects' Awake/Start methods unless script execution order or explicit coordination is configured. NilometerUI.Start deliberately yields one frame before reading avatar selection, implying a dependency on PlayerAvatarSelector.Start. BoatLevelFlow.Start publishes an initial state event; an active listener is needed at that moment to receive it. Verify scene object enablement and script order in Unity when changing initialization behavior.

See event flow, script reference, and scene reference.

Script reference

This inventory includes 55 runtime/support scripts under Assets/scripts and one project editor script under Assets/Editor. Scene attachment statements are based on .meta GUID matches in serialized scenes. “No scene attachment found” does not mean unused: a script may be referenced by a prefab, editor workflow, UnityEvent, or runtime instantiation path.

Core data and messaging

CoreData/BoatLevelEvents.cs — LevelState, SfxId, event structs

Kind and role: Enums and immutable IEvent payloads.

Main API / dependencies: Carries state, collisions, coins, progress, SFX, voice, voice activity, pause/settings messages.

Scene evidence: Indirect; consumed by active boat/Nilometer components.

CoreData/CharacterMood.cs — CharacterMood

Kind and role: Enum.

Main API / dependencies: Expression states used by dialogue and characters.

Scene evidence: Used by Nilometer/character scripts.

CoreData/DialogueLine.cs — DialogueLine

Kind and role: Serializable struct.

Main API / dependencies: Text and optional voice clip data used by dialogue panels.

Scene evidence: Serialized in boat/Nilometer scenes.

CoreData/GameSettings.cs — GameSettings

Kind and role: Static class.

Main API / dependencies: PlayerPrefs properties for music, sounds, voice, VSync, frame cap; Save().

Scene evidence: Read by audio/options systems.

CoreData/Messaging/EventBus.cs — IEvent, EventBus

Kind and role: Interface and static generic event bus.

Main API / dependencies: Subscribe<T>, Unsubscribe<T>, Publish<T> for structs.

Scene evidence: Indirect runtime dependency.

CoreData/Messaging/Service.cs — Service‹T›

Kind and role: Generic struct resolver.

Main API / dependencies: Lazily locates/caches a scene Component; used for BoatLevelFlow.

Scene evidence: Through BoatLevelUI in BoatLevel scene.

Boat level gameplay

GameplaySystems/BoatLevel/BoatLevelFlow.cs — BoatLevelFlow

Kind, ownership, and flow: MonoBehaviour; owns level state, 3 starting stars, coin count, progress, docking, and outcome navigation.

Important behavior/dependencies: CompleteIntro, StartPlaying, OpenHelp, Opensettings, CloseSettings, CompleteArrival, ReplayLevel, LoadNextSection. Registers direct boat/input callbacks. In Update, calculates clamped progress and starts autopilot within configured dock distance.

Scene evidence: Attached to Assets/Scenes/boat/BoatLevel.unity.

GameplaySystems/BoatLevel/BoatController.cs — BoatGame.GameplaySystems.BoatLevel.BoatController

Kind, ownership, and flow: MonoBehaviour; owns 3-lane motion and boat collision response.

Important behavior/dependencies: StepLane, SetProgress, StartAutoPilot, HasReachedAutoPilotStop. Uses Rigidbody2D MovePosition, speed interpolation, lane smoothing, bump/flicker, trigger detection. Publishes lane/blocked/oar events; calls obstacle/coin delegates.

Scene evidence: BoatLevel scene.

GameplaySystems/BoatLevel/BoatInput.cs — BoatInput

Kind, ownership, and flow: MonoBehaviour; translates device input into lane steps.

Important behavior/dependencies: Keyboard A/D/arrows; pointer/touch swipe or screen-half tap; EventSystem raycasts prevent UI taps from steering. IsInputEnabled gates input; raises OnLaneStep.

Scene evidence: BoatLevel scene.

GameplaySystems/BoatLevel/Coin.cs — Coin

Kind, ownership, and flow: MonoBehaviour; collectible visual/state.

Important behavior/dependencies: IsCollected, Collect() and per-frame glow/collection animation. Flow guards duplicate collection.

Scene evidence: No scene GUID match; inspect prefabs/instantiation for use.

GameplaySystems/BoatLevel/Obstacle.cs — Obstacle

Kind, ownership, and flow: MonoBehaviour; obstacle visual/collision feedback.

Important behavior/dependencies: PlayBump; boat controller classifies lane impact and handles response.

Scene evidence: No scene GUID match; inspect prefab instances for use.

GameplaySystems/BoatLevel/CameraFollow.cs — CameraFollow

Kind, ownership, and flow: MonoBehaviour; camera follow and shake.

Important behavior/dependencies: MaxY, Shake, SnapToTarget; flow sets dock limit.

Scene evidence: BoatLevel scene.

GameplaySystems/BoatLevel/BoatLevelVfx.cs — BoatLevelVfx

Kind, ownership, and flow: MonoBehaviour; event subscriber for scene VFX.

Important behavior/dependencies: Handles coin, obstacle, and level state events; uses serialized particle/camera/celebration references.

Scene evidence: BoatLevel scene.

GameplaySystems/BoatLevel/GodRaysPulse.cs — GodRaysPulse

Kind, ownership, and flow: MonoBehaviour; animated ray intensity.

Important behavior/dependencies: Brighten; update pulse driven by serialized values.

Scene evidence: BoatLevel scene.

GameplaySystems/BoatLevel/InfiniteTileStrip.cs — InfiniteTileStrip

Kind, ownership, and flow: MonoBehaviour; background tile recycling.

Important behavior/dependencies: Serialized tile/scroll configuration and transform updates.

Scene evidence: BoatLevel scene.

Boat presentation and shared presentation

Presentation/BoatLevelUI/BoatLevelUI.cs — BoatGame.Presentation.BoatLevelUI.BoatLevelUI

Kind and responsibility: MonoBehaviour; switches intro, tutorial/help, settings, HUD, arrival, and completion views.

Main behavior/dependencies: Button handlers call flow through Service<BoatLevelFlow> and publish UI SFX. Subscribes to state/coin/hit/progress/blocked events. Next handler currently only plays tap feedback; flow call is commented.

Scene evidence: BoatLevel scene.

Presentation/BoatLevelUI/DialoguePanel.cs — namespaced DialoguePanel

Kind and responsibility: MonoBehaviour; boat intro/arrival dialogue.

Main behavior/dependencies: Advances serialized dialogue lines, publishes PlayVoiceEvent, emits finish callback.

Scene evidence: BoatLevel scene.

Presentation/BoatLevelUI/StarsDisplay.cs — StarsDisplay

Kind and responsibility: MonoBehaviour; star display and animation.

Main behavior/dependencies: CountStars, ShowStars; used by boat UI and Nilometer completion.

Scene evidence: BoatLevel and Nilometer scenes.

Presentation/BoatLevelUI/UiMotes.cs — UiMotes

Kind and responsibility: MonoBehaviour; decorative mote particle simulation.

Main behavior/dependencies: Initializes and updates small visual motes.

Scene evidence: BoatLevel scene.

Presentation/BoatLevelUI/UiPulse.cs — UiPulse

Kind and responsibility: MonoBehaviour; pulsing UI animation.

Main behavior/dependencies: Lifecycle starts/stops pulse and updates presentation.

Scene evidence: Main menu, player, map, BoatLevel.

Presentation/SharedUI/DialoguePanel.cs — BoatGame.Presentation.SharedUI.DialoguePanel

Kind and responsibility: MonoBehaviour; reusable timed/voice dialogue.

Main behavior/dependencies: PlayLines, Play, Hide, PreloadTimelines, advance handler; publishes voice/tap events; responds to GamePausedEvent.

Scene evidence: Nilometer scene; separate from boat panel.

Presentation/SharedUI/NilometerUI.cs — global NilometerUI

Kind and responsibility: MonoBehaviour; coordinates Nilometer story, quiz, hints, stars, lesson, and completion.

Main behavior/dependencies: SubmitAnswer validates with WaterOperator; two wrong attempts advance; dialogue callbacks gate water/question progression; selects traveler from avatar. Emits SFX and serialized UnityEvents.

Scene evidence: Nilometer scene.

Presentation/SharedUI/CharacterExpressions.cs — namespaced CharacterExpressions

Kind and responsibility: MonoBehaviour; mood/sprite and talking pose driver.

Main behavior/dependencies: Mood and hop methods; listens to VoiceActivityEvent.

Scene evidence: BoatLevel and Nilometer scenes.

Presentation/SharedUI/DialogueMoodDriver.cs — DialogueMoodDriver

Kind and responsibility: MonoBehaviour; maps dialogue lines to character moods.

Main behavior/dependencies: Hooks dialogue lifecycle and changes character expression.

Scene evidence: BoatLevel scene.

Presentation/SharedUI/OptionsPanel.cs — OptionsPanel

Kind and responsibility: MonoBehaviour; options UI.

Main behavior/dependencies: Opens/closes settings, applies PlayerPrefs-backed audio/display choices, publishes audio settings and pause events.

Scene evidence: Menu, player, map, Nilometer scenes.

Player, persistence, and scene management

Player/player.cs — global player, PlayerGender

Kind and responsibility: MonoBehaviour plus enum; scene-facing player level/step/gender.

Main behavior/dependencies: Start registers with SaveManager; properties and SetGender expose data.

Scene evidence: Player scene.

Player/PlayerData.cs — PlayerData

Kind and responsibility: Serializable data class.

Main behavior/dependencies: Stores level, step, gender and fields consumed by save/avatar systems.

Scene evidence: Runtime data; not a component attachment.

Player/PlayerManager.cs — PlayerManager

Kind and responsibility: MonoBehaviour; older singleton/persistent manager.

Main behavior/dependencies: Awake singleton and DontDestroyOnLoad; source contains minimal Start/Update.

Scene evidence: No active scene GUID match found.

Player/PlayerAvatarSelector.cs — PlayerAvatarSelector

Kind and responsibility: MonoBehaviour; applies saved gender to avatar presentation and reports selection.

Main behavior/dependencies: Start reads SaveManager.CurrentData; ApplyGender, GetCurrentGender.

Scene evidence: Player and Nilometer scenes.

PlayerSelector.cs — class PlayerUISelector

Kind and responsibility: MonoBehaviour; legacy selection buttons.

Main behavior/dependencies: Selects gender through SaveManager; has commented scene-loading call.

Scene evidence: No direct scene GUID match.

PlayerUISelector.cs — class PlayerSelector

Kind and responsibility: MonoBehaviour; selection UI path.

Main behavior/dependencies: Calls SaveManager.SetGender and creates selected player.

Scene evidence: Player scene.

Save system/SaveManager.cs — Exloris.Saving.SaveManager

Kind and responsibility: Persistent singleton MonoBehaviour; owns active and loaded player data.

Main behavior/dependencies: Registers player, loads/saves/deletes, applies gender, creates player; invokes serialized OnLoadedData.

Scene evidence: Main menu, player, Nilometer scenes.

Save system/SaveSystem.cs — SaveSystem

Kind and responsibility: Static persistence service.

Main behavior/dependencies: JSON serialization, AES-CBC encryption, HMAC-SHA256 verification, Base64 envelope, file I/O.

Scene evidence: Called by SaveManager; not attached.

Save system/SaveSecrets.cs — SaveSecrets

Kind and responsibility: Internal static key source.

Main behavior/dependencies: Supplies cryptographic key to save service. Value intentionally omitted.

Scene evidence: Compile-time dependency only; do not expose key.

SceneManager.cs — global SceneManager

Kind and responsibility: Persistent singleton MonoBehaviour; scene load façade.

Main behavior/dependencies: Name/index loads; optional CanvasGroup fade through async loading; reload and next build index.

Scene evidence: Main menu, map, Nilometer scenes.

ManagersDetector.cs — ManagersDetector

Kind and responsibility: MonoBehaviour; button-friendly façade.

Main behavior/dependencies: Forwards scene and sound actions to manager singletons.

Scene evidence: Main menu, player, map, BoatLevel, Nilometer.

SceneOrgniser.cs — SceneOrgniser

Kind and responsibility: MonoBehaviour; invokes serialized start-scene event.

Main behavior/dependencies: Start is the scene entry hook.

Scene evidence: Main menu scene.

Nilometer, audio, localization, and legacy scripts

WaterOperator.cs — global WaterOperator

Kind and responsibility: MonoBehaviour; owns water stage data and sequencing.

Main behavior/dependencies: MoveToLevel, MoveToNextLevel, IsCorrectAnswer, event on reaching target; uses WaterAnimator.

Scene evidence: Nilometer scene.

WaterAnimator.cs — global WaterAnimator

Kind and responsibility: MonoBehaviour; tween-based water-height animation.

Main behavior/dependencies: AnimateYSize and return variants; uses DOTween; kills active tween on destruction.

Scene evidence: Nilometer scene.

Tail.cs — global Tail

Kind and responsibility: MonoBehaviour; question tick/glow sequence.

Main behavior/dependencies: SetGlow overloads and serialized AnimationSequencer list; requires Animation Sequencer runtime symbol.

Scene evidence: Nilometer scene.

CoreGameSystems/Audio/AudioService.cs — namespaced AudioService

Kind and responsibility: MonoBehaviour; music, ambience, SFX, voice, snapshots, mixer settings, voice ducking.

Main behavior/dependencies: Subscribes to audio/state/pause events in enable; builds SfxId lookup; fades with unscaled time; publishes voice activity.

Scene evidence: Main menu scene attachment found; verify lifetime across route.

audio/SoundManager.cs — SoundManager, SoundData

Kind and responsibility: Legacy persistent audio manager.

Main behavior/dependencies: Key-based clip registry and play/stop/loop/mute/localized playback.

Scene evidence: Main menu and Nilometer scenes.

Localization/LanguageSwitcher.cs — LanguageSwitcher

Kind and responsibility: MonoBehaviour; changes active locale and stores language choice.

Main behavior/dependencies: English/Arabic selection and locale code path.

Scene evidence: Player and map scenes.

Localization/LocalizationID.cs — LocalizationID

Kind and responsibility: MonoBehaviour; binds localized text/audio references and responds to table value changes.

Main behavior/dependencies: Exposes string/audio entry IDs; subscriptions in enable/disable.

Scene evidence: Player scene.

Localization/LocalizedInstructionPlayer.cs — same class

Kind and responsibility: MonoBehaviour; selects and plays localized instruction clips.

Main behavior/dependencies: Instruction index, clip/volume selection, playback methods; lifecycle event subscription.

Scene evidence: Player scene.

Localization/LocalizationManager.cs — LocalizationManager

Kind and responsibility: MonoBehaviour; currently contains an instruction list and empty Start/Update methods.

Main behavior/dependencies: Scene attachment alone does not imply active localization behavior.

Scene evidence: Player scene.

Boat/BoatController.cs — global BoatController

Kind and responsibility: Legacy boat MonoBehaviour, distinct from namespaced class.

Main behavior/dependencies: Uses older input/controller flow and movement methods.

Scene evidence: Assets/Scenes/boat/Boat.unity.

Boat/BoatBackground.cs — BoatBackground

Kind and responsibility: Legacy background component.

Main behavior/dependencies: Moves background by distance/step method.

Scene evidence: Legacy Boat scene.

HitDetector.cs — HitDetector

Kind and responsibility: Legacy collision component.

Main behavior/dependencies: Invokes serialized hit/coin UnityEvents.

Scene evidence: Legacy Boat scene.

spawning.cs — spawning

Kind and responsibility: Legacy pooled spawner.

Main behavior/dependencies: Instantiates configured objects and manages active pool entries/return.

Scene evidence: Legacy Boat scene.

ScoreSystem.cs — ScoreSystem

Kind and responsibility: Legacy score MonoBehaviour.

Main behavior/dependencies: Add/subtract/reset score operations.

Scene evidence: Legacy Boat scene.

StarManager.cs — StarManager

Kind and responsibility: Legacy star MonoBehaviour.

Main behavior/dependencies: Star-count/presentation support.

Scene evidence: No scene GUID match found.

Hover/GameObjHover.cs — GameObjHover

Kind and responsibility: World-object hover component.

Main behavior/dependencies: Invoke serialized pointer-enter/exit behavior and restore state.

Scene evidence: Map and mouseHover scenes.

Hover/UIHover.cs — UIHover

Kind and responsibility: UGUI pointer handler.

Main behavior/dependencies: Saves/restores image/text/button visual state on pointer events.

Scene evidence: Map scene.

Hover/HoverTest.cs — HoverTest

Kind and responsibility: Input/hover test helper.

Main behavior/dependencies: Runtime pointer/input probing plus serialized UnityEvents.

Scene evidence: No scene GUID match found.

Editor script

File / type	Kind and responsibility	Evidence / limit
Assets/Editor/WebGLBuildCli.cs — static WebGLBuildCli	Editor-only build command helper.	Relevant to editor/build workflow, not a runtime MonoBehaviour; inspect source and its command entry points before using.

Script usage caveats

- Scene GUID matching confirms serialization in a scene file, not GameObject active state, component enabled state, valid references, callback execution, or inclusion in a particular build after profile processing.
- Prefab references and dynamic use require following prefab YAML and all factory paths; absence from scene YAML is not evidence of dead code.
- Third-party script inventory is excluded from the count. The project imports DOTween, Animation Sequencer, RTLTMPro, UnityEvents 2.0, Cainos assets, and other packages/content.

Gameplay lifecycle

Evidence labels

- **Serialized/configured:** supported by build settings or scene serialization.
- **Code path:** supported by project code, but not runtime-tested.
- **Inferred:** a likely sequence from code/configuration whose actual runtime order or callback binding was not confirmed.

1. Application startup and entry scene

The build settings list `main menu.unity` as the first enabled scene after two disabled entries. Under ordinary Unity build behavior, the first enabled scene is the startup scene. This was not run to verify. The main menu scene serializes a `SceneManager`, `SceneOrgniser`, `SaveManager`, legacy `SoundManager`, `ManagersDetector`, and `AudioService`. Their `Awake`/`Start` relative order is not proven because no explicit execution order or bootstrap coordinator was verified.

`SaveManager.Awake` establishes its singleton/persistent behavior and loads/coordinates player data according to its code. `SceneOrgniser.Start` invokes its serialized start-scene event. Whether this event changes the menu view or scene depends on its scene binding.

2. Menu, player selection, and map route

The enabled scene sequence is menu → player → map → BoatLevel → Nilometer. The menu's serialized button events and `ManagersDetector`/`SceneManager` provide navigation entry points. The player scene contains selection controls, avatar selector, save manager, and localized instruction components. Selection handlers set gender/create or register a player through `SaveManager`; `PlayerAvatarSelector` applies the stored selection. The map scene contains manager and locale/hover components.

The route is supported by enabled scene order and serialized components, but each button-to-scene edge should be verified from its serialized UnityEvent target and by play-testing. A scene being next in build order does not itself transition to it.

3. Boat level initialization

`BoatLevelFlow.Awake` reads the boat start position and sets the camera's maximum Y from the dock reference. `OnEnable` registers callbacks from boat and input; `Start` disables motion/input and publishes `LevelState.Intro`. `BoatLevelUI` subscribes to state changes and uses `Service<BoatLevelFlow>` for button actions. On an Intro event it shows and starts the intro dialogue. The dialogue's finish callback calls `CompleteIntro`, moving the flow to Tutorial.

These are code paths. Whether all required serialized references and listeners are enabled at the relevant moment must be checked in Unity.

4. Starting and steering

The tutorial tap handler publishes a UI tap SFX and calls `StartPlaying`. The flow checks that the state is Tutorial or Paused, restores `Time.timeScale`, enables boat movement and input, and changes state to Playing.

`BoatInput.Update` runs only while enabled. A/D or arrow key presses emit a lane step. Pointer input begins only outside UI; a horizontal displacement beyond the configured inch threshold is a swipe, otherwise release chooses direction based on the press half of the screen. `BoatLevelFlow` forwards the step to `BoatController.StepLane`. The controller clamps lanes to three positions and publishes lane-move audio.

5. Main boat loop and collisions

While Playing or Arriving, BoatLevelFlow.Update computes normalized vertical progress between boat start and dock. It passes progress to the boat speed ramp and publishes progress in increments (or at completion). BoatController.FixedUpdate moves the Rigidbody2D forward, smooths lane position, handles pushback/autopilot, and updates oar sound. Trigger overlaps dispatch coin or obstacle callbacks.

- **Coin:** flow checks IsCollected, calls Collect, increments coins, and publishes coin SFX plus CoinCollectedEvent. HUD updates count; VFX reacts at the event position.
- **Obstacle:** controller classifies first hit and side/forward collision response, then invokes flow callback. Flow publishes bump SFX, removes at most one star on first eligible hit, may publish star-loss SFX, and publishes contact position/stars. UI and VFX respond. A forward collision can block the lane; changing lane clears the block. No separate game-over transition was found when stars reach zero.

6. Pause, help, and settings

Help is modeled as Paused; it sets Time.timeScale = 0 and disables input. Continue calls StartPlaying. Settings also pauses time/input and CloseSettings restores both. BoatLevelFlow.OnDisable restores time scale. The separate shared OptionsPanel publishes GamePausedEvent for dialogue/audio synchronization. Whether every scene's settings overlay uses this exact flow is scene-dependent.

7. Arrival and boat completion

When the boat reaches the configured autopilot distance to the dock, the flow disables input, starts boat autopilot toward the stop point, changes to Arriving, and waits until the controller reaches its stop. After a delay it changes to Arrived. UI then plays arrival dialogue. The dialogue finish callback calls CompleteArrival, publishes win SFX, and sets Complete. UI shows completion stars and coins.

Replay resets time scale and reloads the active scene by build index. The continuation method loads _nextSceneName, defaulting to BoatHistory; that destination is not enabled in current build settings. Further, BoatLevelUI.OnButtonClick_Next currently does not call LoadNextSection, so the completion Next button does not establish the transition in the inspected code.

8. Nilometer learning activity

On Start, NilometerUI waits one frame before resolving the selected traveler, then initializes stars/panels and begins intro dialogue. The one-frame wait is a deliberate ordering workaround for avatar selection, but actual order remains scene-dependent.

After intro, the UI begins level 0. It plays a water-rise prompt; when voice starts, a delay can precede WaterOperator.MoveToLevel. If there is no prompt text, water moves immediately. WaterOperator animates via WaterAnimator, then signals the reached level. The UI opens that question and accepts answers.

- A correct answer disables answers, emits correct SFX, changes traveler/elder mood, animates/highlights the selected answer, then advances dialogue/level logic.
- An incorrect answer increments per-level attempts, subtracts one star, emits wrong SFX, and disables that wrong button. On the first miss, “almost” dialogue enables remaining answers again. On the second miss, “sorry” dialogue moves to the next water level or lesson.
- A hint temporarily disables answers, plays the next hint line, then resumes the question.
- After the final level, lesson and cubit dialogue/cards lead to completion display and remaining stars.

Serialized WaterLevel data, answer button indices, dialogue arrays, optional mood arrays, and callback hookups must be kept aligned. No separate Nilometer failure/game-over state was found.

9. Audio, UI, and visual feedback

AudioService subscribes to typed events and drives sources/mixer/snapshots. It fades music and ducks music/ambience during voice playback using unscaled time. It publishes VoiceActivityEvent for character talking poses. BoatLevelVfx handles coin, impact, and state effects. The Nilometer and legacy scenes also contain SoundManager; confirm which audio component a particular scene uses before changing clips.

10. Persistence, restart, and exit

Player selection and manager methods use SaveManager; SaveSystem serializes PlayerData to JSON, encrypts/authenticates an envelope, and writes player.save under Application.persistentDataPath. Load validates the header and HMAC before decrypting and deserializing; exceptions are logged and yield a null result. Explicit save/load call points are documented in data and dependencies.

The inspected code provides replay and scene loading. A full return-to-menu, app quit, or scene-unload cleanup route was not proven across all scenes. Unity lifecycle methods unsubscribe listeners and restore time scale in several systems, but this analysis did not run the game.

Scenes and transitions

Build scene list

Evidence: ProjectSettings/EditorBuildSettings.asset. There are no separate Build Profile files in the inspected project tree.

0

Scene path: Assets/Exloris/Scenes/Bootstrap.unity

Enabled: No

Notes: Scene file is missing at this path.

1

Scene path: Assets/Exloris/Scenes/FoundationHarness.unity

Enabled: No

Notes: Scene file is missing at this path.

2

Scene path: Assets/Scenes/main menu.unity

Enabled: Yes

Notes: First enabled scene.

3

Scene path: Assets/Scenes/player.unity

Enabled: Yes

Notes: Avatar/player selection.

4

Scene path: Assets/Scenes/Map.unity

Enabled: Yes

Notes: Map navigation.

5

Scene path: Assets/Scenes/boat/BoatLevel.unity

Enabled: Yes

Notes: Namespaced three-lane boat challenge.

6

Scene path: Assets/Scenes/NiloMeter/NiloMeter.unity

Enabled: Yes

Notes: Nilometer learning scene.

No profile-specific scene differences were found because no standalone Build Profile asset was found. Project settings contain platform-specific configuration for Android, iPhone, Standalone, and WebGL; that does not prove equivalent build outputs.

All checked-in scene files

Scene path	Build inclusion	Serialized project-script evidence and role
Assets/Scenes/main menu.unity	Enabled	Scene organization/navigation, save

Scene path	Build inclusion	Serialized project-script evidence and role
		manager, legacy sound manager, manager façade, event audio, and menu presentation components.
Assets/Scenes/player.unity	Enabled	Player component, avatar and selection UI, save manager, localization/instruction helpers, options UI.
Assets/Scenes/Map.unity	Enabled	Scene/manager façade, language switcher, hover components, map UI.
Assets/Scenes/boat/BoatLevel.unity	Enabled	Namespaced BoatLevel flow/input/controller/camera/VFX, boat UI/dialogue/stars, character expression.
Assets/Scenes/NiloMeter/NiloMeter.unity	Enabled	Nilometer UI, water operator/animator, Tail question components, shared dialogue, avatar/expression, stars, save/audio/options helpers.
Assets/Scenes/boat/Boat.unity	Not listed	Legacy global BoatController, background movement, hit detector, score/spawning path.
Assets/Scenes/boat/BoatHistory.unity	Not listed	Title/canvas/camera content; default continuation string points here but scene is not enabled.
Assets/Scenes/SampleScene.unity	Not listed	Camera and 2D light sample.
Assets/Scenes/mouseHover.unity	Not listed	Hover UI/world interaction sample.

Scene YAML GUID matches prove that script components are serialized in these files. They do not establish active state, valid references, runtime instantiation, successful entry, or callback execution. Prefab nesting and all UnityEvent target methods require Inspector/runtime verification.

Navigation mechanisms

Assets/scripts/SceneManager.cs wraps Unity scene loads by name or build index and offers CanvasGroup fade overloads using async loads. It is a persistent singleton. ManagersDetector forwards serialized/UI calls to this manager. Menu/player/map navigation also depends on scene button UnityEvents and string/index targets.

BoatLevelFlow.ReplayLevel reloads the active build index. LoadNextSection loads _nextSceneName by name. The boat UI's Next handler currently only sends tap SFX; its flow call is commented. The default BoatHistory target is not listed as enabled, so this path needs configuration before it can be relied on.

Scene-local setup references

- **BoatLevel:** assign BoatLevelFlow references for boat, input, camera, and dock. BoatLevelUI requires its screens, buttons, dialogue, HUD, progress, and stars. Boat collisions require Rigidbody2D/collider configuration. BoatInput requires an EventSystem for UI exclusion.
- **Nilometer:** WaterOperator needs a WaterAnimator and level entries. NilometerUI needs dialogue, question panels, button/event wiring, star displays, and avatar character references. Level/answer indexes must match.
- **Audio:** AudioService needs sources, mixer exposed parameters, snapshots, clips and an entry for each used SFX ID. Scene-specific legacy SoundManager may coexist.
- **Scene transitions:** scene names must match enabled Build Settings entries. Replay depends on active scene build index.

Profile and runtime limitations

EditorBuildSettings.asset includes Addressables and Input System configuration object references. No distinct profile files were located. Static project data cannot confirm the selected Editor platform, active profile in an external build pipeline, or whether all scene destinations are valid in a produced build.

Data and asset dependencies

Data contracts and persistence

PlayerData is the serializable player model; SaveManager owns the active player and delegates disk operations to SaveSystem. The save file path is Application.persistentDataPath/player.save.

The envelope is Base64 text containing:

EX2S format header | HMAC-SHA256 | AES-CBC IV | AES-CBC ciphertext

Loading checks minimum length and header, verifies HMAC using fixed-time comparison, decrypts, then deserializes JSON. A failure is caught/logged and returns null. Encryption and integrity checks reduce casual modification but do not make an embedded client key a secure trust boundary. SaveSecrets.cs is ignored/local and its secret value is not reproduced here.

GameSettings stores music/SFX/voice volumes, VSync, and frame cap through PlayerPrefs. LanguageSwitcher stores language selection separately. These settings are not the same storage mechanism as player.save.

Localization and audio content

Assets/Exloris/Content/Localization/ contains English and Arabic locale, string table, audio table, shared-data, and localization settings assets. Addressables groups include localization asset/string tables and locale assets. Project scripts use Unity Localization components and locale switching. Direct project-owned Addressables.LoadAssetAsync or equivalent calls were not found in the inspected scripts; localization package internals may load addressable content.

Assets/Audio/ is organized into Ambience, Music, SFX, and Voice folders. AudioService serializes clips/sources/mixer/snapshots and a lookup array keyed by SfxId. Legacy SoundManager uses string keys and clip registrations. Scene serialization and prefabs determine which clips are actually assigned.

Rendering, animation, and UI assets

- Assets/Settings/ contains URP pipeline/renderer/global settings and post-processing profiles.
- Assets/Animation/Avatar/ contains boy/girl sprite animation content.
- Assets/ART/, Assets/UI/, Assets/Atlases/, and imported Cainos content contain art, fonts, and UI materials. File presence does not prove use by active scenes.
- Assets/Resources/DOTweenSettings.asset configures DOTween. WaterAnimator calls DOTween. Tail references Animation Sequencer runtime types and requires the DOTWEEN_ENABLED scripting define for that package's conditional compilation.
- TextMeshPro and RTLTMPro assets support localized typography. Verify actual Arabic direction and layout in the target scenes.

Prefabs

26 prefab files were found in Assets/prefabs/, with names indicating audio, UI/settings, coin/obstacle, island, avatar selector, and manager content. A name is not proof of a runtime role. spawning contains a dynamic Instantiate path, but prefab argument assignment/runtime use was not exhaustively proven. Scene GUID attachments are cataloged in project structure.

Runtime asset loading findings

Search of project-owned runtime/editor scripts found:

- Unity SceneManager.LoadScene and LoadSceneAsync calls in the global scene manager; boat replay/continuation also call Unity scene APIs.
- Instantiate in the legacy spawner.
- No project-owned Resources.Load, Resources.LoadAsync, AssetBundle API, or Addressables load method call.

- One intro video under Assets/StreamingAssets/Exloris/Intro/; a script-level use path was not verified.

This does not rule out package-internal loading, Unity serialized references, UnityEvents, VideoPlayer clip assignment, or prefab-instantiated object dependencies.

Input and physics

BoatInput uses the Input System's Keyboard and Pointer and an EventSystem raycast to ignore UI. The project contains Assets/InputSystem_Actions.inputactions, Assets/BoatControls.inputactions, an Input System package, an EditorBuildSettings config object reference, and legacy ProjectSettings/InputManager.asset. BoatController uses a dynamic, zero-gravity Rigidbody2D with trigger-compatible colliders and MovePosition. Validate target scene colliders and EventSystem module settings in Unity.

External packages and integrations

Package versions below are from Packages/manifest.json; transitive resolved versions belong to Packages/packages-lock.json and may differ. Imported source folders are also present under Assets/.

Relevant Unity packages

Package	Declared version	Project use / evidence
com.unity.render-pipelines.universal	17.6.0	URP asset, 2D renderer/settings assets.
com.unity.inputsystem	1.20.0	Namespaced boat input directly uses Input System keyboard/pointer APIs; action assets are present.
com.unity.localization	1.5.12	Locale, localized string/audio tables, locale switcher components.
com.unity.ugui	2.6.0	Canvas/Button/EventSystem-based screens and controls.
com.unity.test-framework	1.8.0	Package installed; project-owned test source inventory was not established in this pass.
com.unity.recorder	5.1.7	Editor recording tool; no runtime game dependency found.
com.unity.ide.visualstudio	2.0.28	Editor integration.
com.unity.2d.sprite	1.0.0	Sprite asset/editor support.
com.anklebreaker.unity-mcp	Git URL	Unity MCP package; tool/editor integration, not game runtime behavior.
com.kitwright.unity.mcp	Git URL	Unity MCP package; tool/editor integration.
com.coffee.ui-particle	Git URL	UI particle package; project has UI particle-related content/settings. Exact active instances depend on serialized scene/prefab wiring.

Unity engine modules in the manifest include audio, physics2D, UI, animation, JSON serialization, video, and accessibility support.

Imported plugins and source content

- **DOTween:** imported under Assets/Plugins/Demigiant/DOTween; used by WaterAnimator, scene fade animation, and animation components. DOTweenSettings.asset is in Resources.
- **Animation Sequencer:** imported under Assets/AnimationSequencer; Tail.cs references BrunoMikoski.AnimationSequencer.AnimationSequencer. Its runtime type is conditionally compiled by DOTWEEN_ENABLED.
- **RTLTMPro:** imported under Assets/RTLTMPro; supplies RTL TextMeshPro support and Arabic font assets.
- **Unity Events 2.0 for Unity:** imported under Assets/Events 2.0 for Unity; custom Button2 and UnityEvent2 types appear in the Nilometer code.
- **Cainos art/water content:** imported under Assets/Cainos; includes 2D art and Interactive Pixel Water code. Scene/runtime use must be verified per asset.
- **Rounded corners and other UI/plugin assets:** folders are present; usage is scene-specific.

Addressables and localization integration

Addressable settings and localization groups are serialized in `Assets/AddressableAssetsData/`. `EditorBuildSettings.asset` references the Addressables settings object and Input System settings/actions object. Localization tables and locale assets are present. No direct project code calls the Addressables API, so the confirmed integration is configuration/package-backed localization rather than a custom asset-loading service.

Platforms and build integration

Project settings contain platform-specific sections for Android, iPhone, Standalone, and WebGL and platform scripting defines. `Assets/Editor/WebGLBuildCli.cs` is an editor build helper. There are no separate Build Profile assets in the inspected files. This documentation does not assert that builds for all configured targets succeed.

No evidence found

No project-owned network client, multiplayer/networking stack, external analytics SDK, or runtime AI system was identified in the inspected code and manifest. Absence of a direct script reference is not proof that imported package samples or future/uninspected content never uses such systems.

Setup, build, and configuration

Open the project

1. Install Unity 6000.6.0f1 through Unity Hub.
2. Open the verified project root: exloris-2.
3. Allow Package Manager to resolve manifest dependencies. Git-based packages require Git and network access in the Unity environment.
4. Start with Assets/Scenes/main menu.unity, the first enabled scene in the current Editor Build Settings list.

No build or package restore was run during static analysis.

Build scene configuration

ProjectSettings/EditorBuildSettings.asset contains two disabled, missing scene paths followed by five enabled scenes: main menu, player, Map, BoatLevel, and NiloMeter. No separate Build Profile file was found. Confirm the selected target, scene list, and destination scenes in the Unity Editor before building.

The boat flow's serialized/default continuation name is BoatHistory; that scene is not in the current enabled list. The BoatLevel UI Next button's method does not currently invoke flow continuation. Resolve intended continuation before treating the route as complete.

Input configuration

- Input System package: 1.20.0; project input handling includes BoatInput polling Keyboard.current and Pointer.current.
- Input action assets: Assets/InputSystem_Actions.inputactions and Assets/BoatControls.inputactions.
- Legacy axes also remain in ProjectSettings/InputManager.asset.
- ProjectSettings/ProjectSettings.asset indicates active input handling is Input System. Validate EventSystem/UI input module setup in each scene.

Rendering and platform settings

URP 17.6.0 and 2D renderer assets are configured under Assets/Settings. Player settings contain a 1920×1080 default display size and Android/iPhone/Standalone/WebGL-specific configuration. Platform-specific scripting define strings include DOTween/Animation Sequencer support. Treat these settings as configuration evidence, not release validation.

Required scene wiring checks

BoatLevel

- Assign all BoatLevelFlow serialized dependencies: controller, input, camera, dock stop, and next scene.
- Confirm Rigidbody2D, trigger colliders, lane geometry, coin/obstacle components, and camera target.
- Assign BoatLevelUI screens, dialogue panels, buttons, HUD, progress image, and star displays.
- Configure AudioService sources/mixer/snapshots and each used SFX entry; configure BoatLevelVfx references.
- Ensure EventSystem exists so UI taps are filtered by BoatInput.

Nilometer

- Assign WaterOperator, WaterAnimator, ordered water level entries, and question panels.
- Match zero-based answer indexes to each WaterLevel.CorrectAnswerIndex and bind buttons to NilometerUI.SubmitAnswer(levelIndex, answerIndex).
- Align dialogue lines and optional mood arrays. Assign avatar, elder/traveler expressions, star displays, lesson and completion objects.

- Keep Animation Sequencer runtime define active for supported targets that compile Tail.cs.

Build and verification checklist

5. Inspect Unity Console for compile errors and missing serialized references.
6. Confirm the active build target and intended five enabled scenes.
7. Test menu → selection → map → BoatLevel → Nilometer navigation.
8. Test boat tutorial, keyboard and pointer steering, coin, obstacle, help/settings, arrival, completion, replay, and intended Next behavior.
9. Test Nilometer correct answer, first wrong answer/retry, second wrong answer/advance, hints, lesson, and completion.
10. Test Arabic/English UI and voice, save creation/load/corruption handling, and target-device input/layout.
11. Build on the chosen target and test the produced artifact. None of these runtime checks are claimed as completed here.

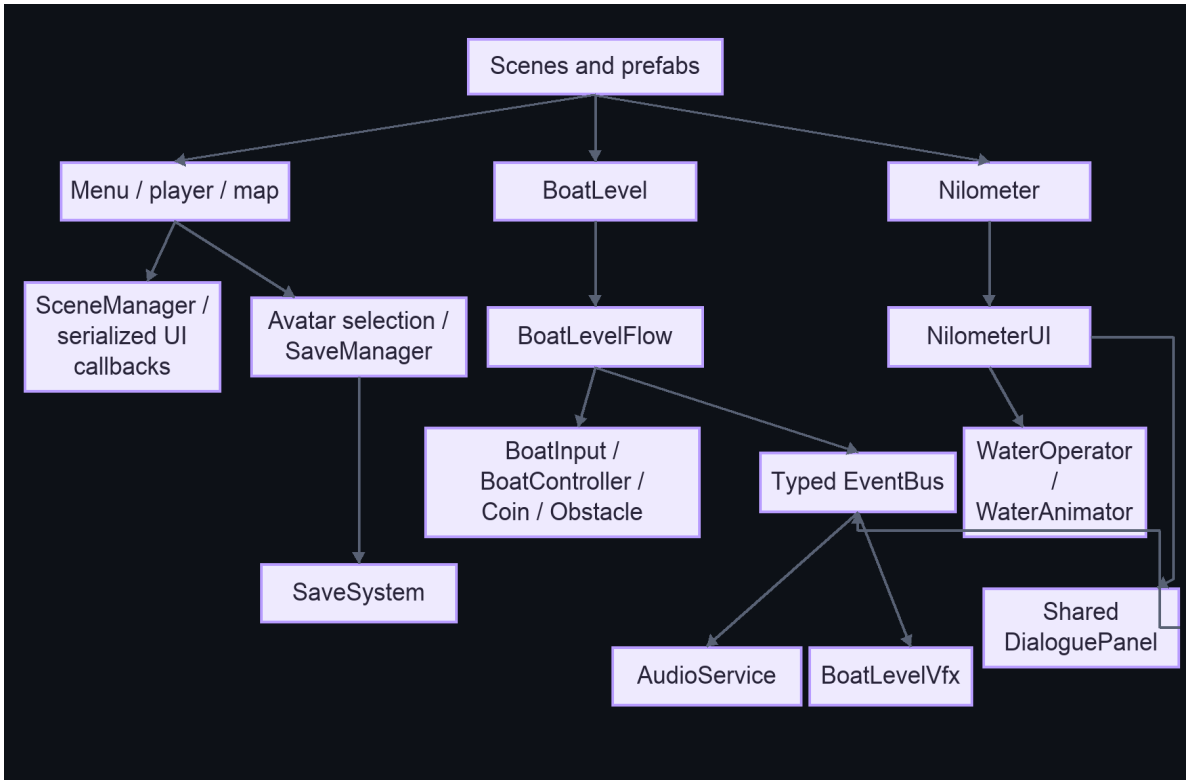
Assets/Editor/WebGLBuildCli.cs exists for WebGL editor builds; inspect its source arguments and usage before invoking. This analysis did not run it.

Editable flowcharts

The diagrams below use Mermaid source and have standalone editable .mmd files in Documentation/diagrams/. They describe source/configuration paths, not play-tested runtime traces.

A. Project overview

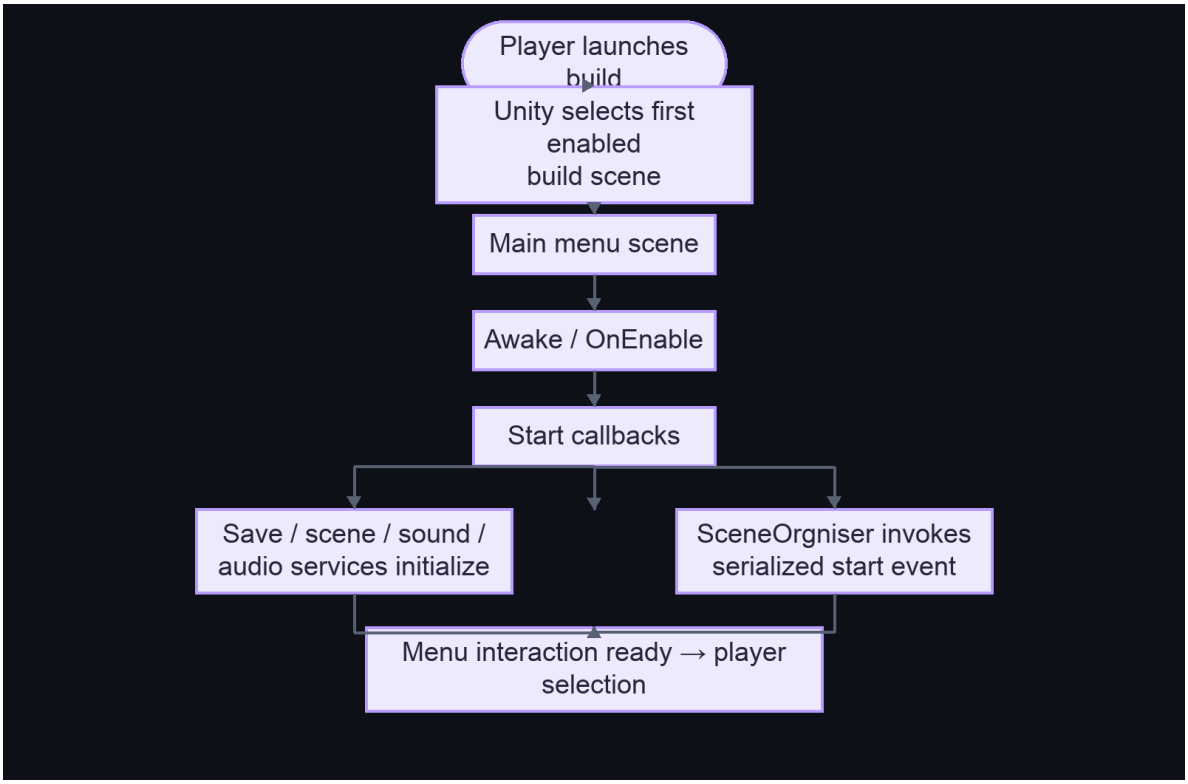
Shows which systems are directly coordinated and which exchange typed events. Based on code references and scene composition.



Source: project-overview.mmd.

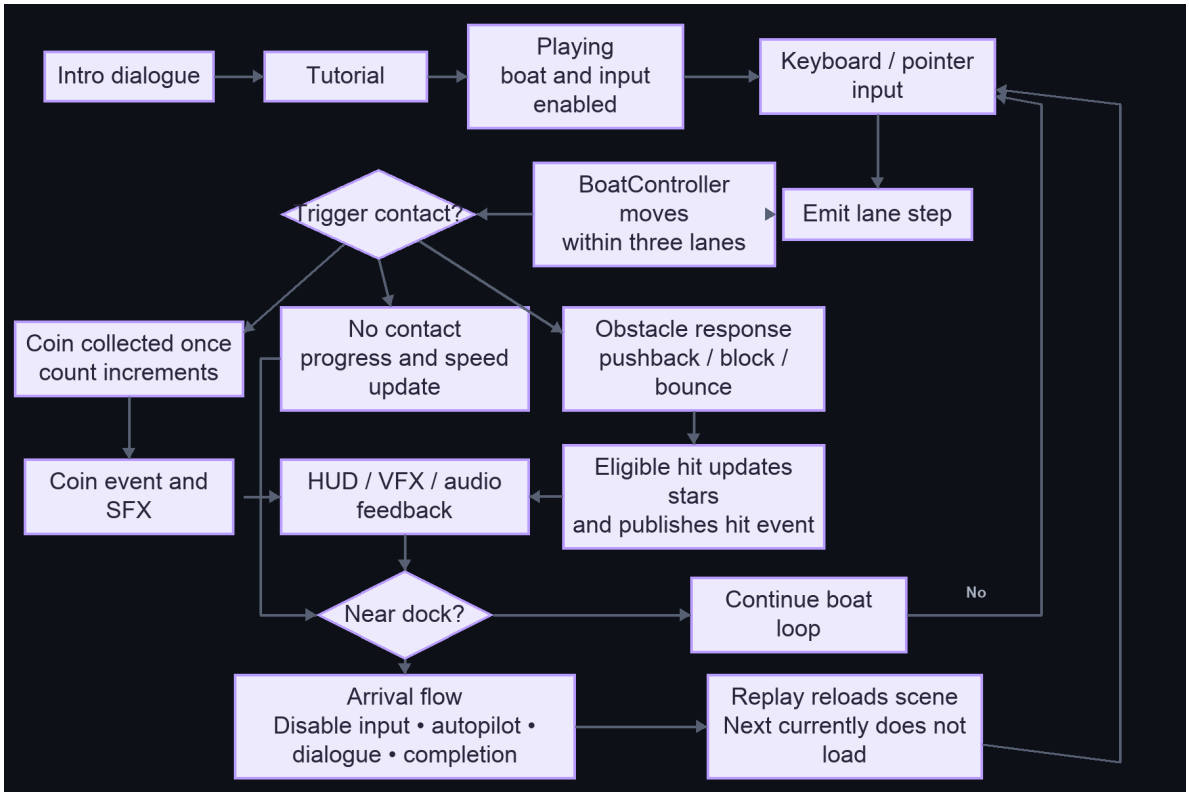
B. Startup and initialization

Shows first enabled scene and known main-menu components. Ordering among unrelated Unity callbacks remains inferred because no runtime trace was collected.



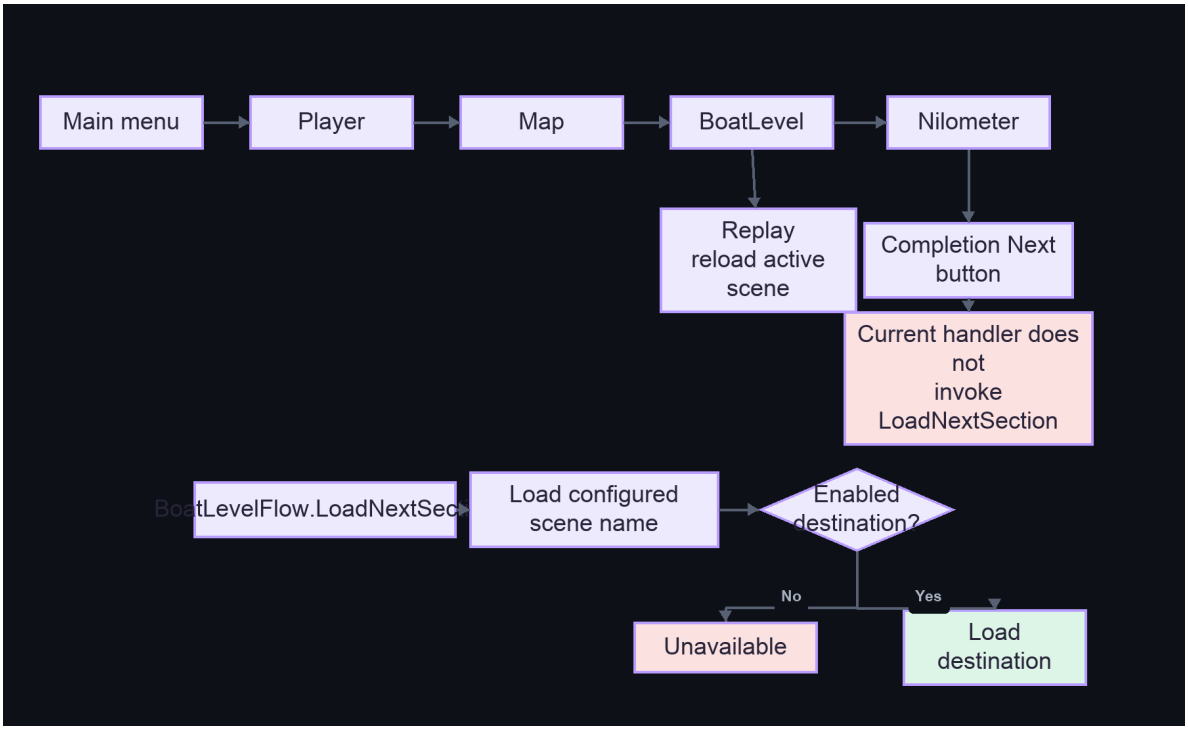
The callback order between objects and the effect of the serialized organizer event depend on Unity scheduling and scene wiring. Source: startup-flow.mmd.

C. Main gameplay loop — boat level



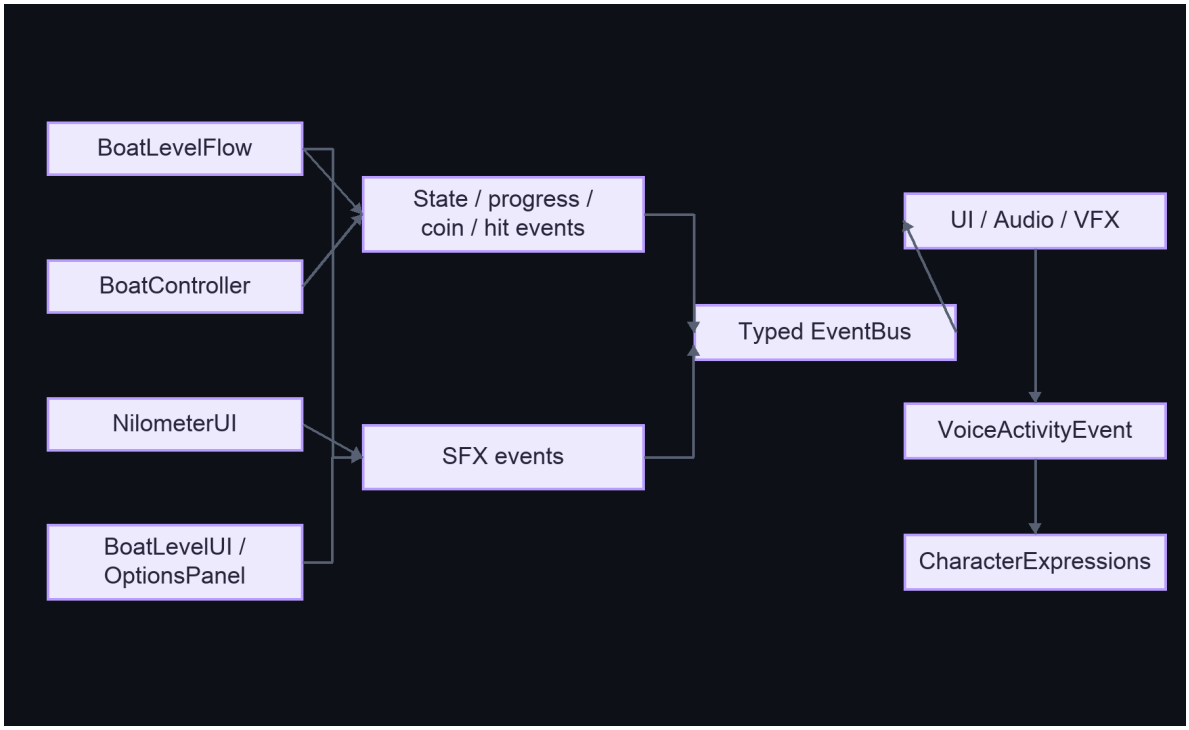
Source: gameplay-flow.mmd. No fail state is shown because none was identified in this state machine.

D. Scene transitions



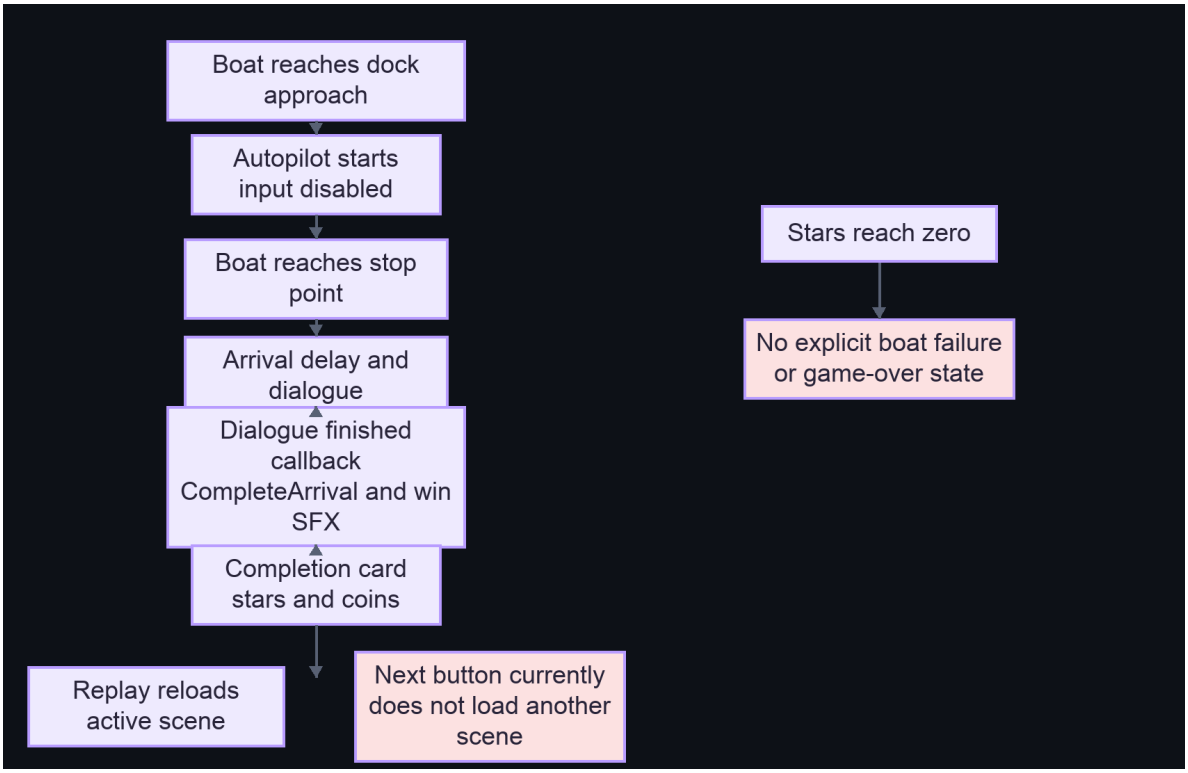
The exact menu/map button edges rely on serialized UnityEvents and should be verified in the scene. Source: scene-transitions.mmd.

E. Typed gameplay event flow



Bus publishing is synchronous; only active subscriptions receive a publish. Source: system-architecture.mmd.

F. Victory, failure, and restart



The Nilometer separately completes after lesson/cubit dialogue and displays remaining stars. No explicit failure screen/state was identified in either flow. Source: gameplay-flow.mmd.

Additional learning and persistence flows

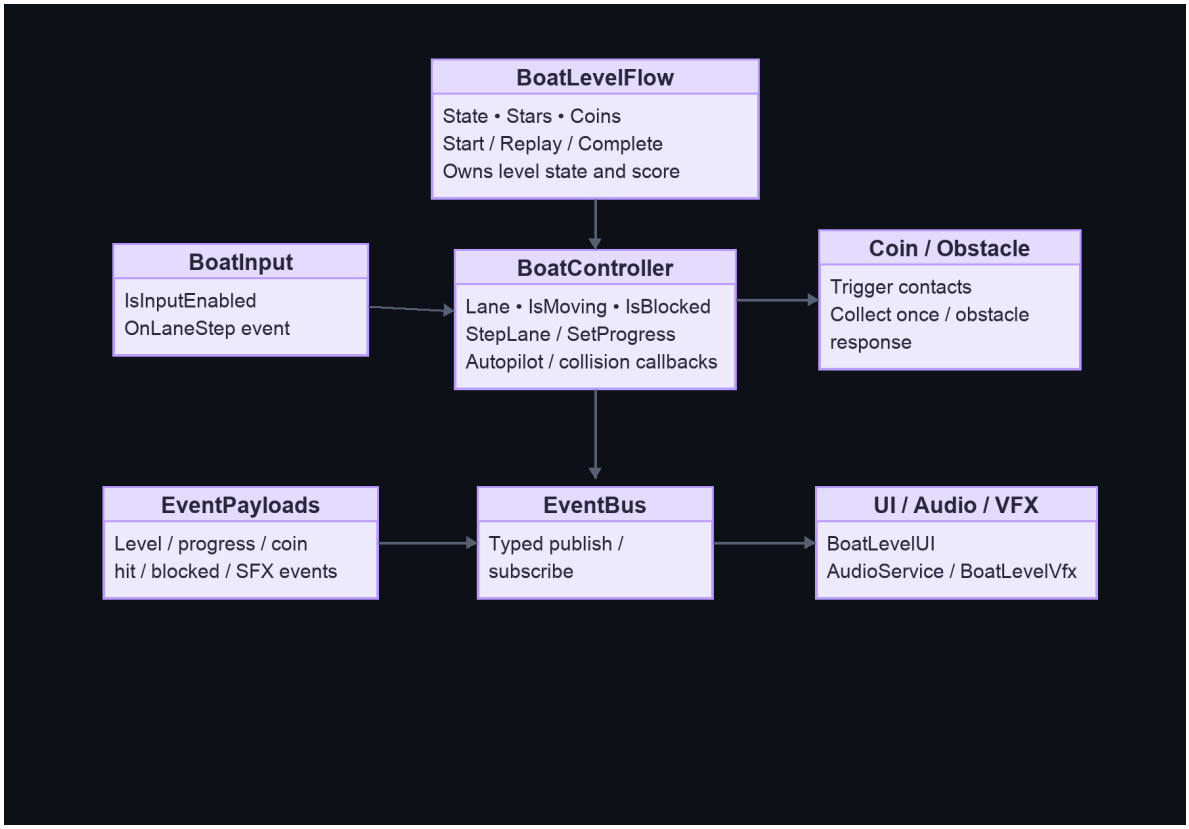
The detailed Nilometer quiz/water flow and save/load envelope sequence are included in the standalone sequence-diagrams.mmd; the save/load static flow is also in class-diagram.mmd where data ownership is summarized. These additional sources are linked from UML diagrams.

UML-style diagrams

The Mermaid diagrams below use code-supported relationships. A serialized field reference is shown as an association/dependency, not composition, unless object lifetime ownership is explicit in code. Diagrams are intentionally limited to system slices rather than every imported class.

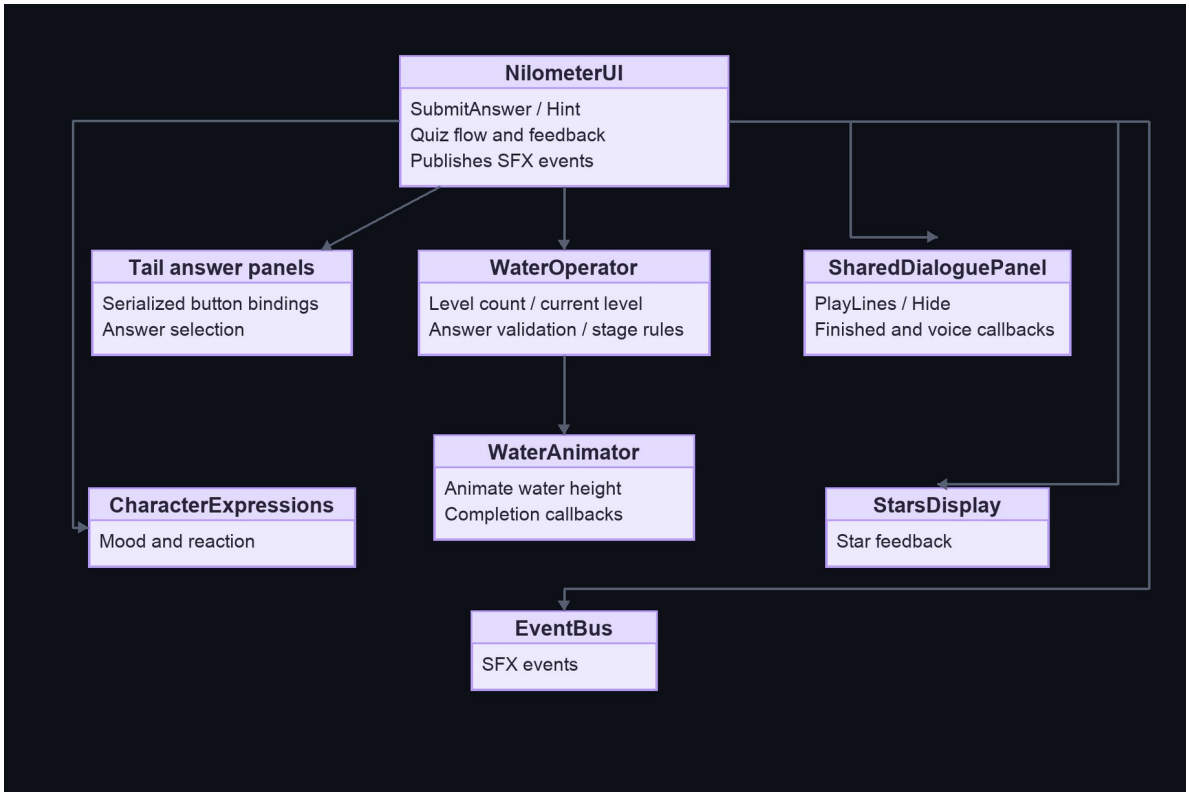
1. Boat system class diagram

Shows the coordinator, input/controller delegates, events, and consumers. It does not show all private tuning fields or every visual helper.



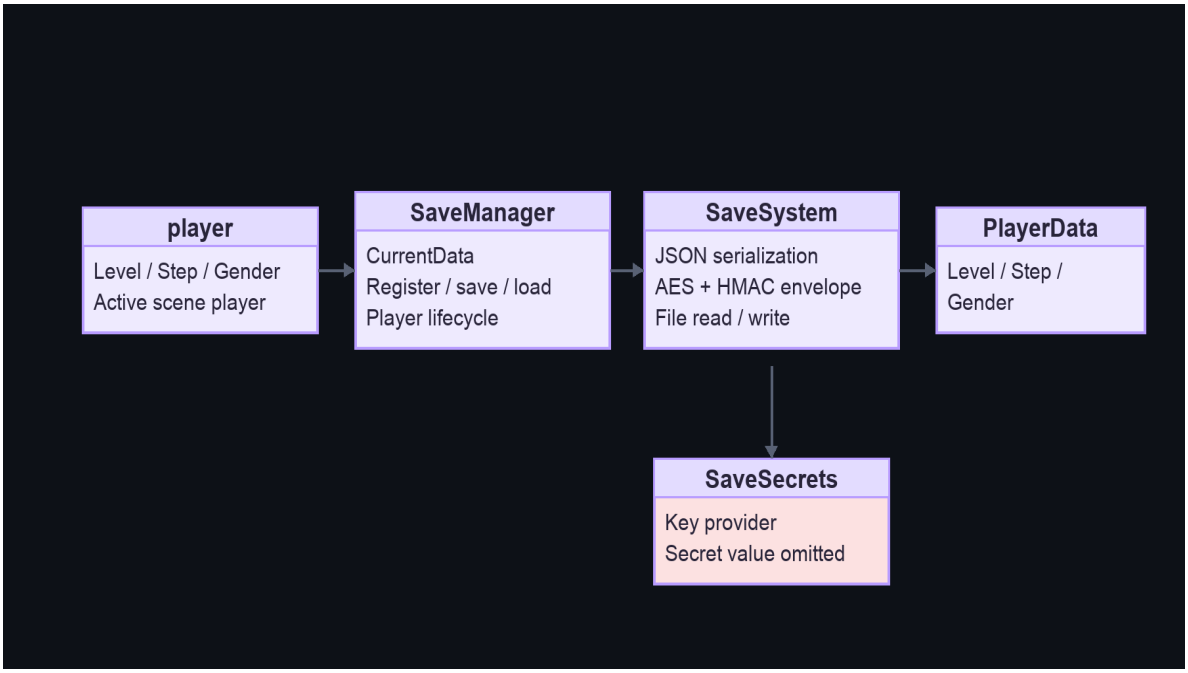
Purpose: document ownership and communication for the namespaced boat feature. `BoatController` invokes flow callbacks; UI/audio/VFX use events. Source: `Assets/scripts/GameplaySystems/BoatLevel/` and `CoreData/BoatLevelEvents.cs`.

2. Nilometer class diagram



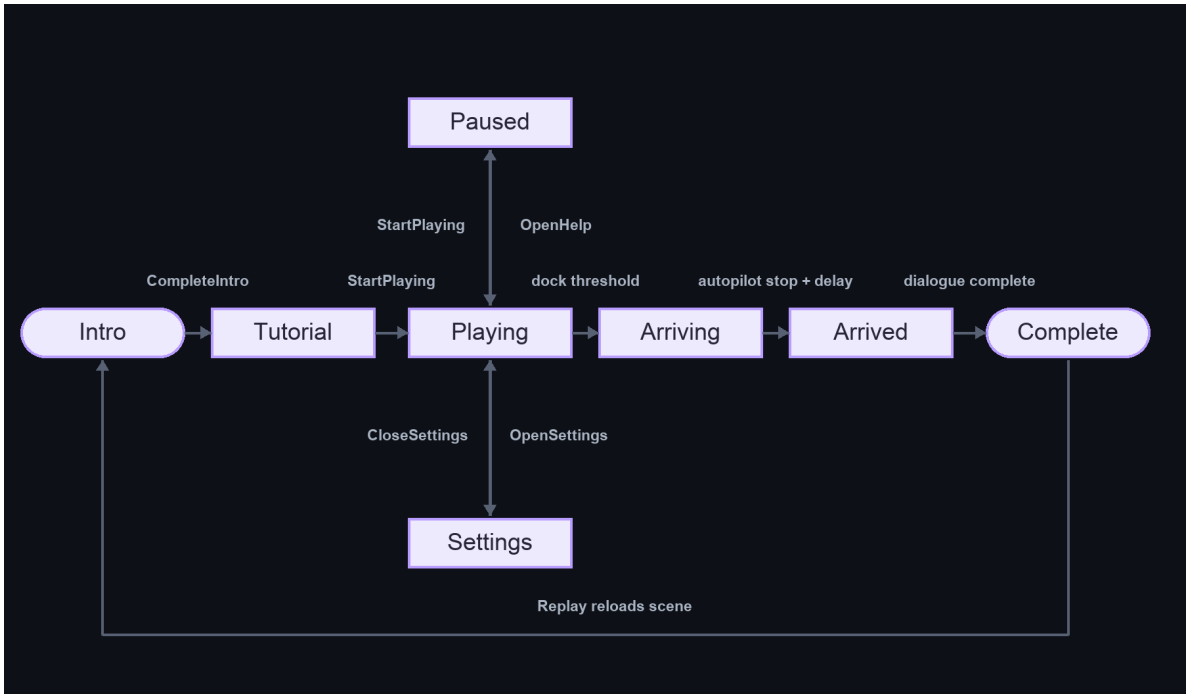
Purpose: show that the UI coordinates quiz state while WaterOperator owns stage rules and water animation. A Tail panel/button binding is serialized; exact per-button wiring should be verified in the scene.

3. Persistence class diagram



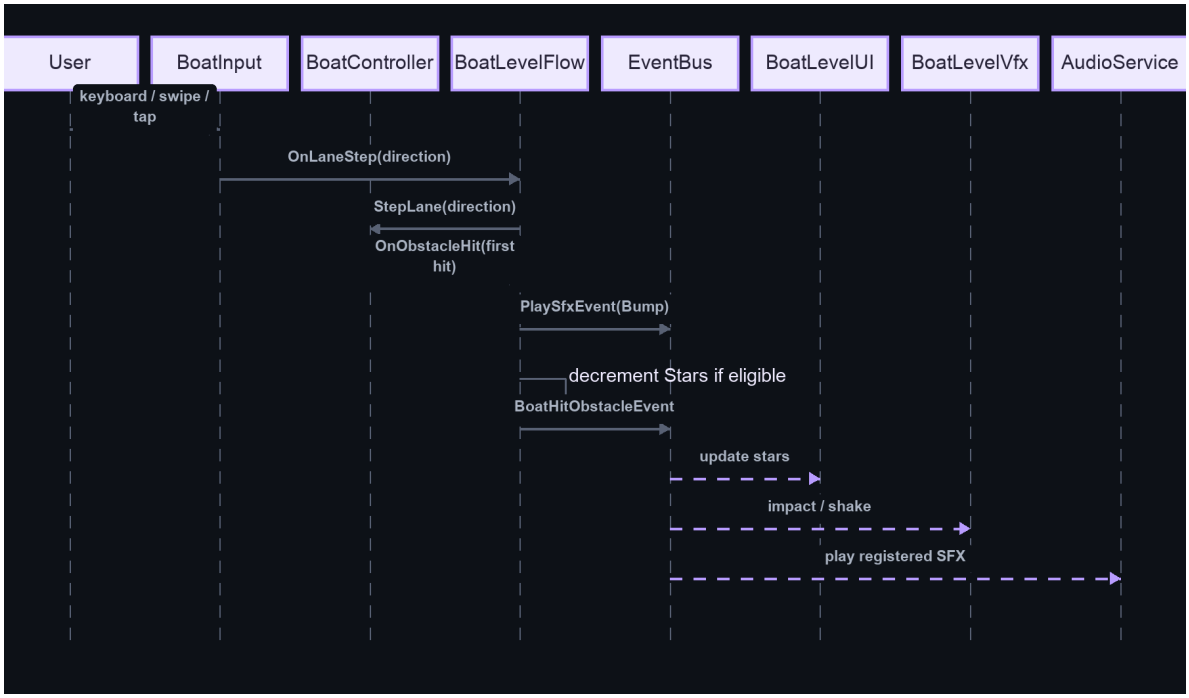
Purpose: clarify player/session ownership versus file serialization. The actual save payload includes the fields serialized by PlayerData; this diagram omits secret contents.

4. Boat state diagram



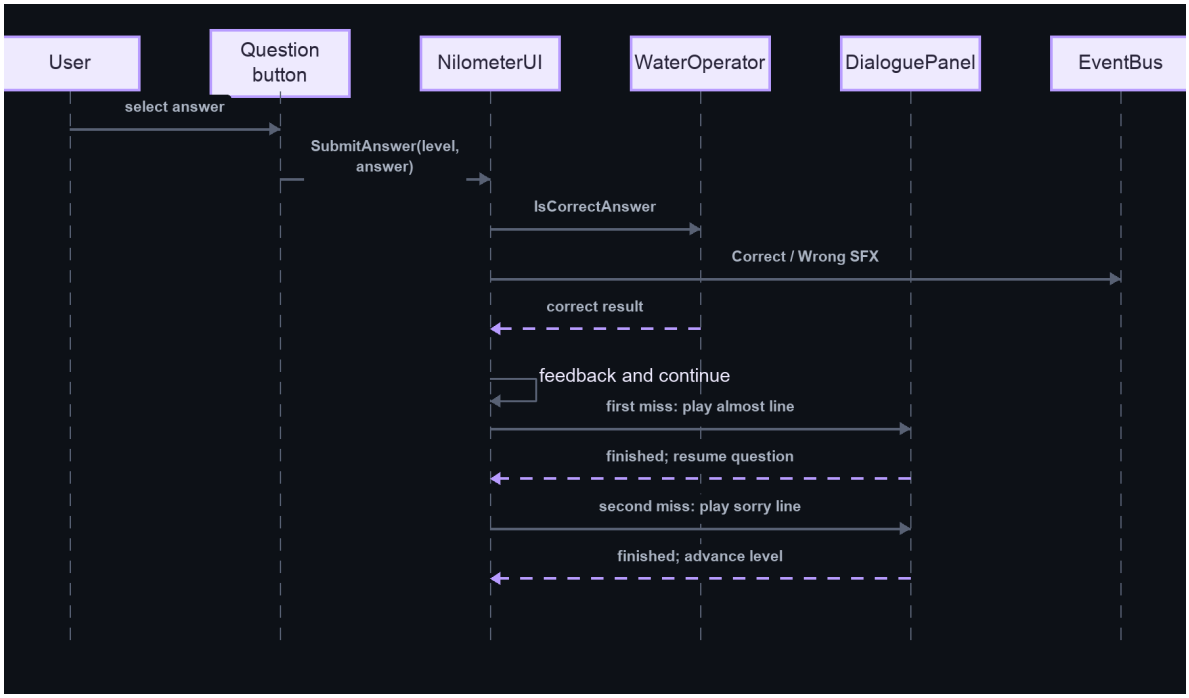
Lesson is present in the LevelState enum but is not entered by the current BoatLevelFlow path. No failure state is defined in this state machine.

5. Sequence diagram — boat obstacle hit



The sequence follows direct C# callbacks and typed subscriptions in the source. Physics timing and scene listener activation were not runtime-tested.

6. Sequence diagram — Nilometer answer



The selected answer index is serialized into button events and validated against current WaterLevel data.

7. Sequence diagram — save/load

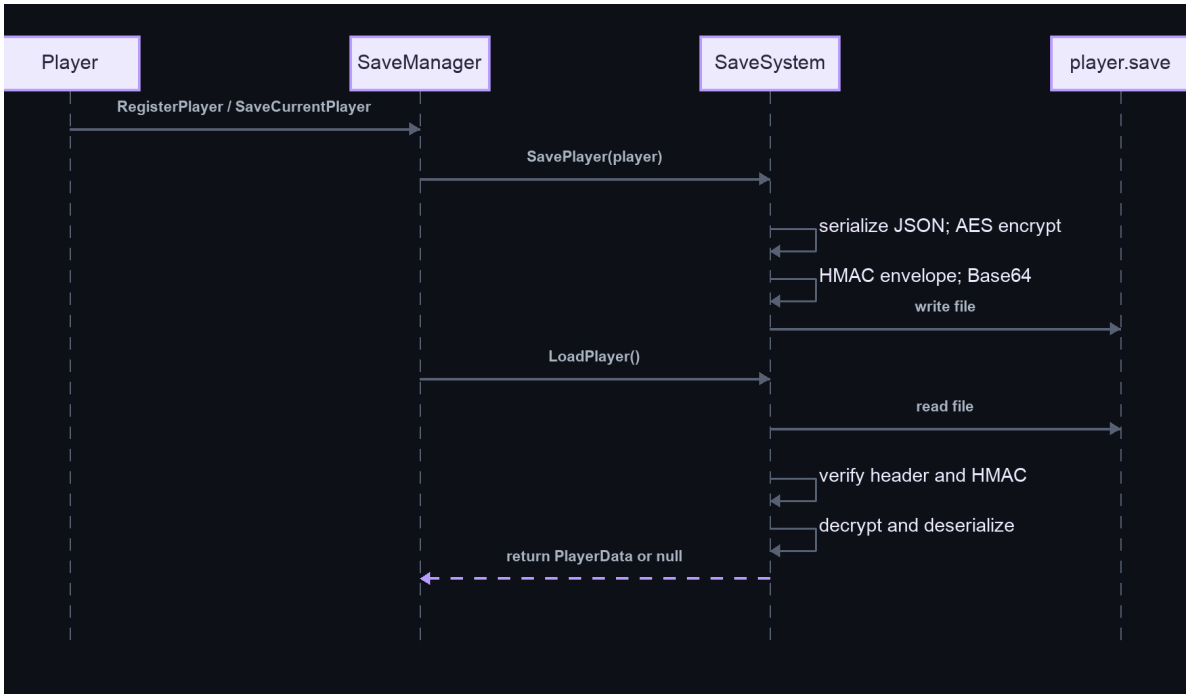


Diagram limits

Mermaid class diagrams cannot show Unity serialized object identity or scene activation states. Diagram lines represent code-level dependencies, callbacks, or serialized references as labeled. Composition is not used unless lifetime ownership is proven.

Glossary and technical notes

Glossary

Term	Meaning in this project
BoatLevelFlow	Namespaced coordinator for boat state, score, progress, docking, and outcome actions.
BoatController	Ambiguous class name: namespaced boat level controller or older global controller in the separate Boat scene.
EventBus	Static synchronous publish/subscribe for IEvent structs in the newer boat/audio architecture.
LevelState	Boat flow state enum: Intro, Tutorial, Playing, Paused, Settings, Arriving, Arrived, Complete, Lesson.
Nilometer	Learning activity coordinated by NilometerUI and water-stage data from WaterOperator.
Tail	Question panel visual component with tick/glow animation sequences.
WaterLevel	Serialized Nilometer stage definition with target height, correct answer, and associated dialogue index.
Addressables	Unity asset delivery/loading system; configuration and localization groups exist, but project-owned scripts inspected do not call its loading API.
confirmed	Directly supported by code or serialized project data inspected. Does not mean play-tested.
inferred	A likely behavior derived from static source/configuration whose runtime execution was not observed.

Important technical notes

- There are two classes named `BoatController` and two named `DialoguePanel`. Use fully qualified names when code namespace resolution could be ambiguous.
- `PlayerSelector.cs` declares class `PlayerUISelector`; `PlayerUISelector.cs` declares class `PlayerSelector`. Serialized component references rely on Unity script GUID/type metadata; renaming files/classes carelessly can break bindings.
- The `LevelState.Lesson` value is defined but not used by the current boat flow. Nilometer lesson state is internal to `NilometerUI`, not that boat enum.
- The boat completion Next UI callback currently does not call the flow's next-scene method. `BoatHistory` is a default string but is not an enabled build scene.
- The save file uses encryption and an HMAC, but the game is client-side and the secret key is part of the application environment. Do not treat this as secure authority for valuable or networked data.
- UnityEvent and inspector bindings are runtime contracts not fully established from C# alone. Scene YAML can reveal serialized object references, but static analysis does not prove the target is active or successfully invoked.

Scope and limitations

- This package is based on static file inspection of the checked-out project, scene YAML, metadata GUIDs, package manifests, and settings. Unity Editor was not opened, the game was not executed, and no build was made.
- There are 56 project-owned C# files in `Assets/scripts` plus `Assets/Editor` (55 runtime/support files and one editor helper). Third-party package source is not fully inventoried.
- Nine checked-in scenes were inventoried. Seven entries exist in Editor Build Settings; two disabled Exloris scene paths are missing. No separate Build Profile assets were found.
- Scene script GUID matches show serialized attachment, not enabled state, successful references, build inclusion, or runtime invocation.
- Prefab files were inventoried by path and selected serialized references. A full nested-prefab and override audit was not performed in Unity.
- Addressables configuration exists; no direct project-owned Addressables load API was found. Package-internal localization behavior was not traced.
- Dynamic asset paths, external build automation, platform-specific runtime behavior, and actual navigation callbacks remain to be verified in Unity.

Suggested follow-up verification in Unity

12. Resolve the missing disabled Bootstrap/FoundationHarness paths or remove them through the project owner's normal workflow.
13. Verify main menu button callbacks and scene transitions against the five enabled scenes.
14. Decide and wire the BoatLevel completion Next behavior and target scene.
15. Inspect Nilometer answer UnityEvents and verify stage/question indexes.
16. Check scene-specific AudioService/SoundManager ownership and persistent singleton behavior.
17. Play through English and Arabic at target aspect ratios and validate save/load on intended platforms.

Appendix Editable Mermaid Sources

Class Diagram

Editable Mermaid source

```
classDiagram
    class IEvent {
        <<interface>>
    }
    class EventBus {
        <<static>>
        +Subscribe~T~(callback)
        +Unsubscribe~T~(callback)
        +Publish~T~(event)
    }
    class LevelStateChangedEvent {
        <<struct>>
        +LevelState Previous
        +LevelState Current
    }
    class BoatLevelFlow {
        +LevelState State
        +int Stars
        +int Coins
        +StartPlaying()
        +ReplayLevel()
        +LoadNextSection()
    }
    class BoatController {
        +StepLane(direction)
        +SetProgress(progress01)
        +StartAutoPilot(stopY)
    }
    class BoatInput {
        +bool IsInputEnabled
        +event OnLaneStep
    }
    class NilometerUI {
        +SubmitAnswer(levelIndex, answerIndex)
        +OnButtonClick_Hint()
    }
    class WaterOperator {
        +MoveToLevel(levelIndex)
        +IsCorrectAnswer(levelIndex, answerIndex)
    }
    class player {
        +SetGender(gender)
    }
    class SaveManager {
        +SaveCurrentPlayer()
        +LoadCurrentPlayer()
    }
    class SaveSystem {
        <<static>>
        +SavePlayer(player)
        +LoadPlayer() PlayerData
    }
    LevelStateChangedEvent ..|> IEvent
    BoatLevelFlow --> BoatController
    BoatLevelFlow --> BoatInput
    BoatLevelFlow ..> EventBus
    BoatController ..> EventBus
    NilometerUI --> WaterOperator
    player --> SaveManager
    SaveManager --> SaveSystem
```

Gameplay Flow

Editable Mermaid source

```
flowchart TD
    Intro[Intro dialogue] --> Tutorial[Tutorial]
```

```
Tutorial -->|Start| Playing[Playing: enable boat and input]
Playing --> Read[Read keyboard or pointer input]
Read --> Lane[Emit lane step]
Lane --> Move[BoatController moves within 3 lanes]
Move --> Collision{Trigger contact?}
Collision -->|None| Progress[Flow updates progress / speed]
Collision -->|Coin| Coin[Collect once; increment count]
Coin --> CoinEvents[Coin event and SFX]
Collision -->|Obstacle| Hit[Pushback / block or bounce]
Hit --> HitEvents[Update stars when eligible; publish hit event]
CoinEvents --> Feedback[HUD / VFX / audio listeners]
HitEvents --> Feedback
Progress --> Dock{Near dock?}
Feedback --> Dock
Dock -->|No| Read
Dock -->|Yes| Arrival[Disable input and start autopilot]
Arrival --> Arrived[Dock stop, delay, arrival dialogue]
Arrived --> Complete[Completion card]
Complete --> Replay[Replay reloads current build scene]
Complete --> Next[Current UI Next handler only plays tap SFX]
Stars[Stars reach zero] -.-> NoFail[No explicit failure state was identified]
```

Project Overview

Editable Mermaid source

```
flowchart TB
  Scenes[Scenes and prefabs] --> Menu[Menu / player / map]
  Scenes --> BoatScene[BoatLevel]
  Scenes --> NileScene[Nilometer]
  Menu --> Navigation[SceneManager / serialized UI callbacks]
  Menu --> Player[Avatar selection / SaveManager]
  BoatScene --> Flow[BoatLevelFlow]
  Flow --> Actors[BoatInput / BoatController / Coin / Obstacle]
  Flow --> Bus[Typed EventBus]
  UI[BoatLevelUI] --> Flow
  UI --> Bus
  Bus --> Audio[AudioService]
  Bus --> VFX[BoatLevelVfx]
  NileScene --> Quiz[NilometerUI]
  Quiz --> Water[WaterOperator / WaterAnimator]
  Quiz --> Dialogue[Shared DialoguePanel]
  Dialogue --> Bus
  Player --> Save[SaveSystem]
```

Scene Transitions

Editable Mermaid source

```
flowchart LR
  Menu[main menu] --> Player[player]
  Player --> Map[Map]
  Map --> Boat[boat/BoatLevel]
  Boat --> Nile[NiloMeter/NiloMeter]
  Boat --> Replay[Reload active scene]
  Boat --> NextHandler[Completion Next button]
  NextHandler -->|Current code| NoLoad[Does not invoke LoadNextSection]
  FlowNext[BoatLevelFlow.LoadNextSection] --> Name[Load configured _nextSceneName]
  Name --> Check{Enabled scene available?}
  Check -->|No| LoadProblem[Destination unavailable]
  Check -->|Yes| Destination[Load destination]
```

Script Dependencies

Editable Mermaid source

```
flowchart LR
  subgraph BoatFeature[Boat level direct dependencies]
    Input[BoatInput] -->|OnLaneStep| Flow[BoatLevelFlow]
    Flow -->|StepLane / progress / autopilot| Boat[BoatController]
    Boat -->|coin / obstacle delegates| Flow
    Flow -->|Service resolver| UI[BoatLevelUI]
  end
  subgraph EventConsumers[Typed event consumers]
    Flow --> Bus[EventBus]
```

```
  Boat --> Bus
  UI --> Bus
  Bus --> Audio[AudioService]
  Bus --> VFX[BoatLevelVfx]
  Audio --> Expressions[CharacterExpressions]
 end
 subgraph NileFeature[Nilometer direct dependencies]
  Nile[NilometerUI] --> Water[WaterOperator]
  Water --> Animator[WaterAnimator]
  Nile --> Dialogue[Shared DialoguePanel]
  Nile --> Tail[Tail answer panels]
 end
 subgraph Saving[Player persistence]
  Player[player] --> Manager[SaveManager]
  Manager --> Save[SaveSystem]
  Save --> Secret[SaveSecrets key provider]
 end
```

Sequence Diagrams

Editable Mermaid source

```
sequenceDiagram
    participant User
    participant Input as BoatInput
    participant Boat as BoatController
    participant Flow as BoatLevelFlow
    participant Bus as EventBus
    participant UI as BoatLevelUI
    participant VFX as BoatLevelVfx
    participant Audio as AudioService
    User->>Input: keyboard / swipe / tap
    Input->>Flow: OnLaneStep(direction)
    Flow->>Boat: StepLane(direction)
    Boat->>Flow: OnObstacleHit(obstacle, isFirstHit)
    Flow->>Bus: PlaySfxEvent(Bump)
    opt eligible hit and stars remain
        Flow->>Flow: decrement star
        Flow->>Bus: PlaySfxEvent(StarLost)
    end
    Flow->>Bus: BoatHitObstacleEvent(stars, contact)
    Bus-->>UI: update HUD stars
    Bus-->>VFX: impact response
    Bus-->>Audio: play SFX

    rect rgb(238, 246, 255)
        participant User2 as User
        participant Question as Tail button
        participant Nile as NilometerUI
        participant Water as WaterOperator
        participant Dialogue as Shared DialoguePanel
        User2->>Question: select answer
        Question->>Nile: SubmitAnswer(level, answer)
        Nile->>Water: IsCorrectAnswer(level, answer)
        alt correct
            Nile->>Nile: feedback and continue flow
        else wrong first attempt
            Nile->>Dialogue: play almost line
            Dialogue-->>Nile: finished; resume question
        else wrong second attempt
            Nile->>Dialogue: play sorry line
            Dialogue-->>Nile: finished; next level or lesson
        end
    end

    rect rgb(244, 255, 244)
        participant Player as player
        participant Manager as SaveManager
        participant System as SaveSystem
        participant File as player.save
        Player->>Manager: register / save
        Manager->>System: SavePlayer(player)
        System->>System: serialize, encrypt, HMAC, Base64
        System->>File: write
        Manager->>System: LoadPlayer()
```

```
    System->>File: read
    System->>System: verify header and HMAC
    alt valid
        System->>System: decrypt / deserialize
        System-->>Manager: PlayerData
    else invalid
        System-->>Manager: null and log error
    end
end
```

Startup Flow

Editable Mermaid source

```
flowchart TD
    Launch[Player launches build] --> Enabled[Unity uses first enabled build scene]
    Enabled --> Menu[main menu scene]
    Menu --> Awake[Components receive Awake / OnEnable]
    Awake --> Start[Components receive Start]
    Start --> Services[Save / scene / sound / audio components initialize]
    Start --> Organizer[SceneOrgniser invokes serialized start event]
    Services --> Ready[Menu interaction available]
    Organizer --> Ready
    Ready --> Selection[User starts and enters player selection]
```

System Architecture

Editable Mermaid source

```
flowchart TB
    Scene[Scenes / serialized references] --> Flow[BoatLevelFlow]
    Input[BoatInput] -->|lane step callback| Flow
    Flow -->|direct calls| Boat[BoatController]
    Boat -->|collision callbacks| Flow
    Flow --> Bus[EventBus]
    Boat --> Bus
    BoatUI[BoatLevelUI] -->|flow methods via Service| Flow
    BoatUI <--> Bus
    Bus --> Audio[AudioService]
    Bus --> VFX[BoatLevelVfx]
    Bus --> Voice[VoiceActivityEvent]
    Voice --> Character[CharacterExpressions]
    Nile[NilometerUI] --> Water[WaterOperator / WaterAnimator]
    Nile --> Dialogue[Shared DialoguePanel]
    Player[Player / avatar selection] --> SaveManager[SaveManager]
    SaveManager --> SaveSystem[SaveSystem]
    SaveSystem --> File[player.save]
```